POZNAN UNIVERSITY OF TECHNOLOGY

Depth Camera Model for RGB-D Dataset Generation and Training Neural Network

by

Maciej Brąszkiewicz, Tomasz Naklicki

in the
Institute of Robotics and Machine Intelligence
Faculty of Control, Robotics, and Electrical Engineering

Supervisor: Dominik Belter, Ph.D, D.Sc. (Eng.)

June 29, 2025

Streszczenie

We współczesnej robotyce coraz większe znaczenie zaczyna mieć wykorzystywanie sieci neuronowych oraz algorytmów uczenia maszynowego, które wymagają znaczących zbiorów danych do nauki. Popularnym sposobem zbierania dużych ilości próbek jest generowanie ich w sposób syntetyczny, często przy pomocy prostych, idealnych modeli sensorów. Celem niniejszej pracy jest utworzenie modelu kamery RGB-D, który poprawi jakość generowanych danych, tak aby były bardziej podobne do zdjęć robionych prawdziwą kamerą oraz pozwalały na lepsze działanie wyuczonych na nich sieci w warunkach rzeczywistych.

W ramach pracy zaproponowano dokładny model kamery RGB-D pozwalający na generowanie syntetycznych danych oraz ich obróbkę w celu symulowania zjawisk pojawiających się na rzeczywistych zdjęciach. Zdjęcia zostały wygenerowane przy pomocy aplikacji Blender. Aby zapewnić podobieństwo syntetycznych danych do prawdziwych obrazów, zostały dodane takie zjawiska jak braki w pomiarach głębi, szum pomiarowy i błędy projekcji.

W celu zweryfikowania poprawności stworzonego modelu przeprowadzono eksperymenty wykorzystujące sieci neuronowe U-Net. Testy zostały przeprowadzone na rzeczywistych danych zebranych przy pomocy kamery Orbbec Femto Mega. Otrzymane wyniki jednoznacznie wykazują, że zaproponowany model jest wartościowy i pozwala na poprawę jakości generowanych danych względem danych z idealnego modelu. Wskazuje to, że nauka sieci na wzbogaconych danych pozwala na lepsze działanie w warunkach rzeczywistych.

Abstract

In modern robotics, the importance of neural networks and machine learning algorithms is steadily increasing. This technology requires substantial datasets for training. A popular approach to collecting large numbers of samples involves generating them synthetically, often using simple, idealized sensor models. The aim of this work is to develop an RGB-D camera model that enhances the quality of generated data so it resembles images captured by a real camera more closely, enabling trained networks to perform better under real-world conditions.

This thesis proposes a detailed RGB-D camera model that facilitates the generation of synthetic data and their processing to simulate phenomena observed in real images. The images were generated using the Blender application. To ensure similarity between synthetic data and real-world imagery, phenomena such as depth measurement dropouts, measurement noise, and projection errors were introduced.

To validate the proposed model, experiments were conducted using U-Net neural networks. The tests were performed on real data collected with the Orbbec Femto Mega camera. The obtained results clearly demonstrate that the proposed model is effective and improves the quality of generated data compared to data from an idealized model. This indicates that training networks on enriched data enables better performance under real-world conditions.

Contents

Streszczenie i

Abstract ii

1 Introduction 1
- 1.1 Aim . . . 1
- 1.2 Motivation and goals of the thesis . . . 2
- 1.3 Thesis organization . . . 2
- 1.4 Division of work . . . 3

2 State of the art 4
- 2.1 Existing solutions overall . . . 4
- 2.2 Measurement Noise . . . 4
- 2.3 Camera Calibration . . . 6
- 2.4 Modeling camera distortions . . . 6
- 2.5 Use of Synthetic Training Data in Neural Network Training . . . 9

3 Proposed system 11
- 3.1 Processing pipeline . . . 12
  - 3.1.1 3D representation . . . 12
  - 3.1.2 Data generation . . . 14
    - 3.1.2.1 *Perfect* camera model . . . 15
    - 3.1.2.2 Proposed model . . . 16
  - 3.1.3 Simulating occlusions . . . 17
    - 3.1.3.1 Difference in FOVs . . . 18
    - 3.1.3.2 Angle of reflection . . . 19
    - 3.1.3.3 Color of the simulated objects . . . 20
  - 3.1.4 Simulating measurement noise . . . 23
  - 3.1.5 Projection of the depth image into RGB space . . . 24
- 3.2 Novelty of the proposed system . . . 26

4 Experimental evaluation 27
- 4.1 Experimental setup . . . 27
- 4.2 Experiments . . . 29
  - 4.2.1 Comparison between neural network trained on data from the proposed camera model and the *perfect* camera model . . . 29
  - 4.2.2 Comparison of the performance of the neural networks for different λ parameter values . . . 30
  - 4.2.3 Comparison of the performance of the neural networks on data generated with the the *perfect* camera model . . . 32
  - 4.2.4 Ablation study of the proposed depth camera model . . . 34

4.3 Conclusions 36

5 Conclusions **37**
5.1 Summary 37
5.2 Conclusions 39
5.3 Future work 39

Bibliography **40**

Chapter 1

Introduction

Recent advancements in vision systems, sensors, and artificial intelligence have significantly contributed to the progress in robotics. One of the most important technologies currently used in this field are neural networks, which have had a particularly strong impact on image processing. A key challenge in using neural networks is the large amount of data required for training. Acquiring sufficient data is often complex and resource-intensive. The collected data may not cover the full diversity of the scenes, lightning conditions and variations of objects placement in the scene for thousands of images necessary to create a robust dataset. Annotating objects on the images is also a time-consuming process. For this reason, the ability to artificially generate training data is highly valuable, as it allows the creation of rich datasets with fewer resources, and without relying on physical sensors. Synthetic data is not without its limitations, however. Using unmodified depth data from a rendering environment may lead to the neural network struggling on real-world, imperfect data.

In robotics, RGB-D cameras are commonly used for visual data acquisition, as they provide not only color images but also depth maps, which allow for precise distance estimation to objects. A crucial step in enabling dataset generation in this way is the preparation of an accurate RGB-D camera model that reflects the errors and imperfections present in real-world data. The chosen approach to reconstructing camera characteristics included the simulation of noise, distance related limitations, depth-RGB misalignment, and light reflection artifacts. Due to the imperfections of the device, the generated image is also distorted. In this work, the Orbbec Femto Mega camera was used.

1.1 Aim

The main goal of the paper is preparing the environment for the generation of synthetic RGB-D images, using the model based on the Orbbec Femto Mega device, and the creation and implementation of an advanced RGB-D camera model by simulating real sensor inaccuracies. The work includes a comparison of the characteristics of the proposed model with those of Orbbec Femto Mega. It is then evaluated using synthetic datasets, one based on a simple pinhole camera

model and one based on the proposed model. The evaluation consists of a comparison between neural networks trained on these datasets. Specific steps of the process have been outlined:

- creating a synthetic scene in Blender,
- creating a simple camera model based on Orbbec Femto Mega parameters,
- recreating the geometric transformation between RGB and depth sensors,
- modeling real camera inaccuracies,
- creating datasets, one based on a simple pinhole camera model and one based on the proposed model,
- training neural networks using prepared datasets,
- comparing the networks, using real images collected with Orbbec Femto Mega.

1.2 Motivation and goals of the thesis

The main motivation of this paper is to create the synthetic camera model which would allow to replace the real sensor in the task of training dataset generation and avoid its limitations. In many robotics and computer vision applications, the quality and diversity of training data significantly impact the robustness and generalization of neural models. However, real sensors often introduce device specific artifacts and constraints that limit data variability and control over experimental conditions. By developing a synthetic RGB-D model that accurately reflects sensor imperfections while offering flexibility in scene generation, this work addresses the need for scalable, consistent, and easily produced datasets that can better support learning-based systems in complex environments. Such a model would be useful in many different tasks involving the usage of neural networks trained on depth data, such as object detection, camera-guided robot control etc.

1.3 Thesis organization

The MSc thesis is divided into the following chapters:

- Introduction - presenting the problem and the goals of the paper.
- State of the art - a review of currently available literature concerning the subject.
- Proposed system - description of the data generation framework and the image modifications present in the model.
- Experimental evaluation - evaluation of the proposed approach.
- Conclusions - presenting conclusions from the work and discussing possible future work.

1.4 Division of work

The work was divided into 5 chapters:

- Introduction - Brąszkiewicz, Naklicki
- State of the art - Brąszkiewicz, Naklicki
- Proposed system
  - 3D representation - Brąszkiewicz
  - Data generation - Brąszkiewicz, Naklicki
  - Simulating occlusions - Naklicki
  - Simulating measurement noise - Naklicki
  - Projection of the depth image into RGB space - Brąszkiewicz
  - Novelty of the proposed system - Brąszkiewicz, Naklicki
- Experimental evaluation
  - Experimental setup - Brąszkiewicz, Naklicki
  - Experiments
    - Comparison between neural network trained on data from the proposed camera model and from the *perfect* camera model - Brąszkiewicz
    - Comparison of the performance of the neural networks for different λ parameter values - Naklicki
    - Comparison of the performance of the neural networks on data generated with the *perfect* camera model - Naklicki
    - Ablation study of the proposed camera model - Brąszkiewicz
  - Conclusions - Brąszkiewicz, Naklicki
- Conclusions
  - Summary - Brąszkiewicz
  - Conclusions - Naklicki
  - Future work - Brąszkiewicz, Naklicki

Chapter 2

State of the art

2.1 Existing solutions overall

Many studies have been conducted to present methods for eliminating the limitations of RGB-D cameras, so that the data they collect is not distorted. The conclusions presented in many of these works can also help in modeling these limitations. A key factor to simulate are measurement noise, light beam reflections, camera distortions, and offset between RGB and depth camera. Measurement noise can be divided into several types:

- thermal noise,
- a low number of photons reaching the photosensitive cell,
- noise increasing with distance from the camera,
- edge noise,
- noise caused by external factors, such as sunlight,
- noise caused by type and color of the surface.

To obtain a well-functioning model, one can attempt to simulate some of these phenomena.

2.2 Measurement Noise

In the study *Modeling Kinect Sensor Noise for Improved 3D Reconstruction and Tracking* [24], the noise present in data from the Microsoft Kinect v2 camera was categorized into two types: axial noise, which increases with the distance from the camera, and lateral noise, which appears on the edges of surfaces and grows with the increase of the angle between the camera and the given plane. Noise values were determined through measurements of several white sheets of paper with various known sizes, positioned at different distances and angles relative to the

sensor. Based on these measurements, mathematical noise models for this camera were derived. The following formula for axial noise 2.1 was defined:

$$\sigma_z(z,\theta) = 0.0012 + 0.0019(z-0.4)^2 \tag{2.1}$$

Its application is limited to the range $10° < \theta < 60°$. For angles approaching 90° a modification 2.2 was introduced:

$$\sigma_z(z,\theta) = 0.0012 + 0.0019(z-0.4)^2 + \frac{0.0001}{\sqrt{z}} \cdot \frac{\theta^2}{(\frac{\pi}{2}-\theta)^2} \tag{2.2}$$

The formula for the lateral noise 2.3 was described as:

$$\sigma_L(\theta)[px] = 0.8 + 0.035 \cdot \frac{\theta}{\frac{\pi}{2}-\theta} \tag{2.3}$$

Mallick *et al.* in [21] classified the types of noise encountered when using the Kinect v1 camera. They also presented methods for mathematically modeling these types of noise, based on geometry, empirical data, and statistics. Additionally, they introduced a distinction between *spatial* noise, which appears within a single image frame, and *temporal* noise, observable across multiple consecutive frames. Repeating this process can allow for the reconstruction of noise characteristics in measurements taken with other cameras. The following formulas were also presented:

$$\sigma_z = (9.0 \cdot Z^2 - 26.5 \cdot Z + 20.237) \cdot 10^{-3} \tag{2.4}$$

$$\Delta Z = (2.73Z^2 + 0.74Z - 0.58) \cdot 10^{-3} \tag{2.5}$$

Equation 2.4 describes the *depth precision* parameter, while 2.5 defines the *depth resolution*.

Fiedler and Mueller [15] observed the influence of environmental factors, such as temperature, on the quality of calibration and depth measurement using the Kinect camera. This is due to the changing parameters of the sensor. It was noted that at higher temperatures, the captured image became more blurred and slightly magnified. Relationships between depth measurements and the temperature of the Kinect sensor were determined, demonstrating a positive correlation.

Barron and Malik [8] utilized lighting and shape detection in RGB images to determine depth. The noisy IR camera data was used in a complementary manner, supporting shape identification and more accurate distance estimation. This method requires only a few correct readings within a given object to determine its position, which helps mitigate the effects of noise and dead pixels on measurement quality.

Berdnikov et al. [10] categorized occlusions present in depth images into two types: those occurring at the edges of foreground objects and those caused by surface characteristics such as reflectivity. The observations presented in their work regarding the nature of occlusions are useful for modeling such effects in camera simulations.

Certain algorithms for noise modeling have already been developed. One such noise model for Kinect camera is available at [5]. The implementation is based on the paper [8] by Barron and Malik. However, its use is not exactly suitable for newer cameras, such as Orbbec Femto Mega,

because it is based on a different depth measurement technique - Kinect uses structured light, Femto Mega uses ToF (AMCW).

2.3 Camera Calibration

The impact of misalignment errors between cameras can be mitigated through proper device calibration, which can be achieved in various ways. The importance of performing this process correctly is emphasized in [16]. It has been observed that most methods rely on understanding the noise characteristics present in the raw sensor data. Wei *et al.* in [31] proposed a calibration method for the Azure Kinect based on using the same checkerboard for both RGB and IR cameras. The images were captured from different positions, angles, and distances, which allowed for more accurate determination of both intrinsic and extrinsic parameters. The data was then processed to align both images.

Zhang and Zhang, in their paper [33], noted that traditional checkerboard-based calibration methods are unreliable for RGB-D cameras, as the depth camera often struggles to detect corners. Their proposed solution is based on the principle of maximum consistency, which includes two main ideas:

1. The checkerboard points in the depth image should lie on a common plane (the plane is known from the RGB camera calibration).
2. Additional corresponding points in the depth and RGB images can be selected manually or automatically to improve calibration accuracy.

Herrera *et al.*, in [13], proposed a method for simultaneous calibration of the RGB camera, the depth camera, and their relative positions. The process only requires images of a flat surface captured from different angles and does not rely on detecting discontinuities in color or depth, which increases its robustness to measurement noise. The RGB camera is calibrated using the standard checkerboard-based method. Initial depth parameters are estimated based on the manually selected corner positions on the flat surface. The plane is identified in both images, and the geometric shift between the cameras is calculated. Calibration is performed in such a way as to minimize the squared errors between the measured corner positions and the transformed images from both cameras.

2.4 Modeling camera distortions

One of the possible approaches to modeling distortions is to use a pinhole camera model as a foundation for further modifications. It is a mathematical representation of camera's operation, assuming that light passes through a single point. Using appropriate projection methods, it is possible to project a 2D image into a 3D point cloud and then project it back onto a 2D plane.

The basic parameters required for such a projection include:

- focal lengths (f_x, f_y),
- coordinates of the image center (c_x, c_y),
- rotation matrix,
- translation vector.

With proper camera calibration, it is possible to obtain its intrinsic properties, which are extremely useful when creating the model. Gang Peng *et al.* in [26] highlight the importance of accurately determining distortion parameters, which can then be incorporated into the creation of a more precise camera model. Lelowicz, in [19], presents a modified pinhole camera model that includes radial and tangential distortions caused by lens imperfections.

$$\begin{bmatrix} u_u \\ v_u \\ 1 \end{bmatrix} = m \begin{bmatrix} d_1 & 0 & v_0 \\ 0 & d_2 & u_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & x \\ r_{21} & r_{22} & y \\ r_{31} & r_{32} & z \end{bmatrix} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} \tag{2.6}$$

The above model (2.6) was derived by assuming camera calibration on a flat surface and transforming the classical pinhole camera model with the coordinate $Z = 0$. The study describes and compares various distortion models. It was concluded that the key aspect is the ability to mathematically invert the distortion removal operation, as it enables more precise model construction. However, this operation becomes more complex when a larger number of distortion parameters are involved. One way to represent radial distortion is the division model, described by equations 2.7 and 2.8.

$$u_u = \frac{u_d - u_0^d}{1 + k_1 r_d^2 + k_2 r_d^4 + k_3 r_d^6 + ... + k_n r_d^{2n}} \tag{2.7}$$

$$v_u = \frac{v_d - v_0^d}{1 + k_1 r_d^2 + k_2 r_d^4 + k_3 r_d^6 + ... + k_n r_d^{2n}} \tag{2.8}$$

- (u_d, v_d) i (u_u, v_u) are corresponding pairs of points in images with and without distortion,
- (u_0^d, v_0^d) is the centre of distortion,
- r_d is the distance of the distorted point from the centre of distortion given by the equation 2.9.

$$r_d^2 = u_d^2 + v_d^2 \tag{2.9}$$

The study notes that in most cases, a single-parameter division model is sufficient to accurately model radial distortions. Its advantage in the context of applying distortions to the camera is its relatively easy invertibility, which involves solving a second-degree polynomial equation. Tangential distortions are related to the misalignment of the lens with respect to the image plane and are described by equations 2.10 and 2.11.

$$u_d = p_1(r_u^2 + 2u_{ud}^2) + 2p_2 u_{ud} v_{ud} \tag{2.10}$$

$$v_d = p_2(r_u^2 + 2v_{ud}^2) + 2p_1 u_{ud} v_{ud} \tag{2.11}$$

where

- $u_{ud} = u_d - u_0^d$
- $v_{ud} = v_d - v_0^d$

Watanabe and Saito, in [30], emphasize the significant influence of lenses on camera distortions compared to the pinhole camera model and propose a fuzzy modeling method for such sensors. The parameters of the camera equation are selected based on the distance from the sensor. This approach allowed them to reduce both the mean absolute error and standard deviation between the pixel positions in the real and modeled camera images by more than half. The article *Three Camera Lens Distortion Correction Models and Its Application* [29] compares the performance of three popular methods for modeling distortions:

- Brown's model,
- polynomial model,
- model based on Fourier series.

Brown's model is represented by two sets of equations:

$$\begin{cases} \Delta x_r = \bar{x}(k_1 r^2 + k_2 r^4 + k_3 r^6) \\ \Delta y_r = \bar{y}(k_1 r^2 + k_2 r^4 + k_3 r^6) \end{cases} \tag{2.12}$$

$$\begin{cases} \Delta x_d = p_1(3\bar{x}^2 + \bar{y}^2) + 2p_2 \bar{x}\bar{y} \\ \Delta y_d = p_2(\bar{x}^2 + 3\bar{y}^2) + 2p_1 \bar{x}\bar{y} \end{cases} \tag{2.13}$$

Equation 2.12 describes the correction of radial distortions, while 2.13 addresses the correction of tangential distortions. Luber and Reulke in [20], describe a polynomial camera model and its practical implementation in various types of sensors. They pointed out the inaccuracy of this method for cameras with very wide or very narrow fields of view. The study also proposes a method to minimize these inaccuracies.

2.5 Use of Synthetic Training Data in Neural Network Training

An accurately modeled camera is a crucial component for generating data that adequately reflects data collected in real-world conditions. The impact of artificial datasets on training neural networks is widely discussed in the literature. Marcu and Grava [22] emphasize the importance of both the quality and quantity of data in neural network training. They describe methods of data augmentation in cases where collecting a large dataset is difficult. This process involves modifying existing data through various transformations. Other techniques that allow for expanding an insufficiently large dataset are also presented.

Bothma and Luwes [12] described a study on the use of synthetically generated data for image processing tasks. They created a dataset using images of electronic components (resistor, capacitor, SMD capacitor) modeled in Blender, which were then augmented. The F1-score was used to evaluate classification performance. For accurately modeled components (resistor and capacitor), F1-scores of 0.720 and 0.729 were achieved, respectively. For the less accurately modeled SMD capacitor, the score was only 0.205. This study demonstrates the importance of proper object modeling.

Jeon *et al.* [17] demonstrated the effectiveness of synthetic data in training networks for autonomous driving tasks. They used the CARLA simulator and compared results with real-world data from the Waymo dataset. They also observed that the best results were obtained when synthetic and real data were used in a 1:1 ratio.

In their study, Varga and Bunke [28] conducted an experiment comparing a handwriting recognition system trained on an existing dataset of handwritten text with one extended by synthetically generated text lines. The results showed that instead of adding more naturally collected data, synthetic data could be added to achieve comparable learning outcomes.

Dong *et al.* [14] used synthetically generated false data for adversarial training. Their method positively impacted the network's performance and robustness against errors.

Yu *et al.* [32] evaluated human detection quality in image frames depending on the type of data used. They conducted an experiment comparing real data, synthetic data, and a combination of both. It was shown that using the combined dataset yielded the best results. A network trained only on synthetic data performed worse than one trained on real data. This phenomenon was likely caused by the inaccurate modeling of real people. A second experiment was also conducted, where increasing amounts of synthetic images were added; however, it was observed that beyond a certain point, performance improvements plateaued.

Belov *et al.* [9] used synthetic data to train a network for recognizing flying vehicles - a task where collecting a diverse and extensive real-world dataset is difficult. The synthetic data, based on existing 3D models, was added to the dataset and significantly improved the network's performance.

Zhang *et al.* [34] used synthetic image generation for training a neural network for 6-degree-of-freedom pose estimation. Images were rendered by combining a 3D object model with 2D background images. Depth data was estimated from a generated RGB image, and uniform Gaussian and Speckle noises were added. Their network was tested on both synthetic and real-world evaluation data. On synthetic images, they achieved average precision of 98%, which dropped to 68% while analyzing real data. One of the possible reasons why the results change so drastically is inaccurate depth imagery.

Khan *et al.* [18] explored how synthetically generated 3D face models can be used to construct a high accuracy ground truth for depth. To render the images, Blender was used. Imported characters were put on the scene with cameras and lights placed in a fixed position. Using *Cycles* render engine, RGB image and corresponding depth map were obtained. Four different neural network models were used to compare the results, including two pre-trained depth estimation models: DepthDense and MiDas. All networks were tested on the synthetically generated data. The results show that this method has the potential to generate vast datasets of synthetic facial images that can be used for the neural network training.

Lately, the Isaac Sim [6] provided by NVIDIA became a popular tool for generating synthetic data. The application allows for simulating many sensors, including a depth camera. It was used in many recent works, including [11], [23], [25] or [27]. The depth sensor simulation in Isaac Sim, while allowing for simulating camera parameters and lens distortion, does not include any other phenomena occurring on real-world data. This leaves an incomplete recreation of a sensor, which may negatively affect the performance of the generated depth data in real life conditions.

Chapter 3

Proposed system

The aim of this study is to prepare an RGB-D camera model for use in generation of synthetic training data. The proposed model should provide methods of recreating phenomena observed on real depth data from those sensors, including noise, occlusions etc. The created dataset will be used to train a neural network. The final outcome will be a comparison of the performance of a neural network trained on data augmented by the proposed camera model, versus one trained on data rendered from a *perfect* model with no further post-processing. The test is conducted using data from a real camera.

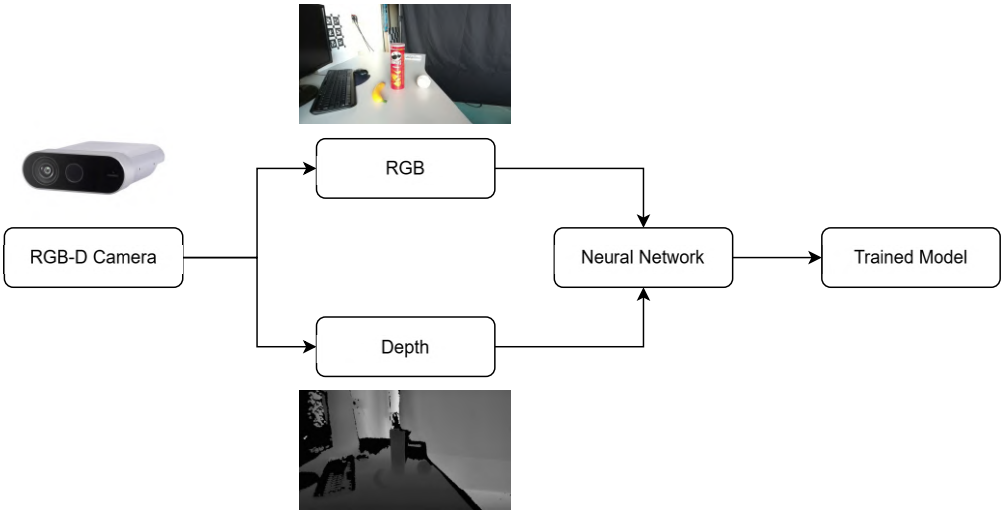


FIGURE 3.1: General system layout using a real camera

Fig. 3.1 illustrates a general process of training a neural network using data collected from a real camera.

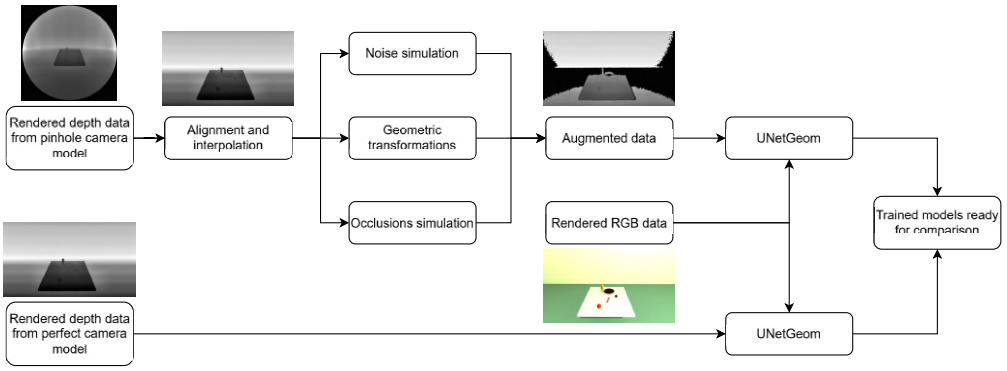


FIGURE 3.2: General system layout using the proposed model

Fig. 3.2 presents the proposed process of training neural network using synthetically generated data.

3.1 Processing pipeline

Processing pipeline consists of the following steps:

- modeling a 3D representation of the scene using external software (Blender),
- generation of input data in the form of RGB and depth images (*Rendered depth data from pinhole/perfect camera model* 3.2),
- simulating occlusions based on distance from the camera, angle of incidence and colors of the objects (*Occlusions simulation* 3.2),
- simulating measurement noise (*Noise simulation* 3.2),
- projection of depth image into RGB space and interpolation (*Geometric transformations* 3.2).

The full model is available in project repository [3].

3.1.1 3D representation

Creating a scene image requires the use of external software. For this purpose, Blender [1] was used. Blender is a free, open source application, which includes a full set of 3D tools used for modeling, texturing, rendering and animation. It provides a wide range of resources for modeling realistic 3D scenes, as well as a functioning Python API, allowing for automation of most tasks. For the purpose of this paper, the following functionality was used:

- Python API, for controlling the objects with a simple Python script,
- compositing nodes, used for generating depth images with specific pixel values,

- materials and texturing, allowing for accurate recreation of real scenes,
- customized light source, used to create specific lighting conditions,
- *Cycles* render engine for generating depth image using GPU.

A set of objects from the YCB dataset [7] was placed within the scene. The YCB dataset is a standardized benchmark dataset widely used in robotics and computer vision. It was developed to provide a set of real world objects for neural network training and evaluating machine learning algorithms. It consists of high resolution RGB-D images and 3D mesh models of items commonly seen in American shops. Several scenes were created. Fig. 3.3 and fig. 3.4 show how a real scene could be visualized in Blender.

Figure 3.3: Real scene

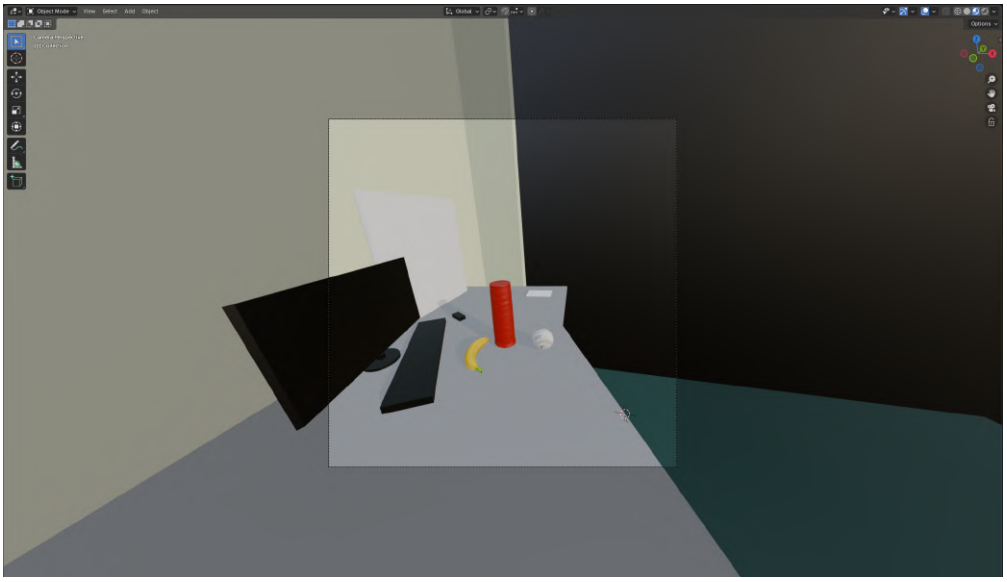


Figure 3.4: Scene recreated in Blender

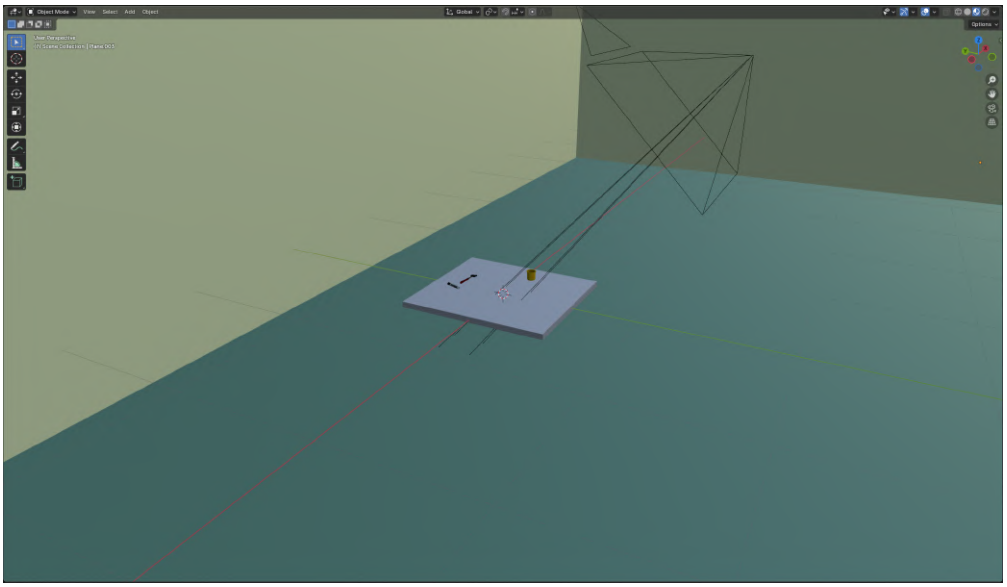


FIGURE 3.5: Scene used in creating the dataset

Dataset was created using different scene, which consisted of simple table placed in the middle of the room (Fig. 3.5). Objects from YCB dataset were randomly picked and placed on table for each render.

3.1.2 Data generation

The input data consists of two different representations:

- RGB image,
- depth image.

In a real camera, they are captured by two separate, internal sensors. In the case of artificially generated data, the scene is modeled in Blender, and pairs of images (RGB and depth) are rendered. Each image pair is captured from a different position and angle, providing diversity in the collected data.

To acquire depth data, Blender's *Compositing* settings were used, as seen in fig. 3.6. The following nodes were used:

- Render Layers - used to acquire the metric distance measured by the sensor,
- Multiply - data was multiplied by 1000, to emulate the internal multiplier used in Orbbec Femto Mega camera,
- Divide - data was divided by 65535, to counteract the multiplication present while exporting Blender images to 16-bit PNG (saving process in Blender includes scaling the image to 0-65535 range),
- Composite - output node, allowed for rendering the images using the *Compositing* settings,

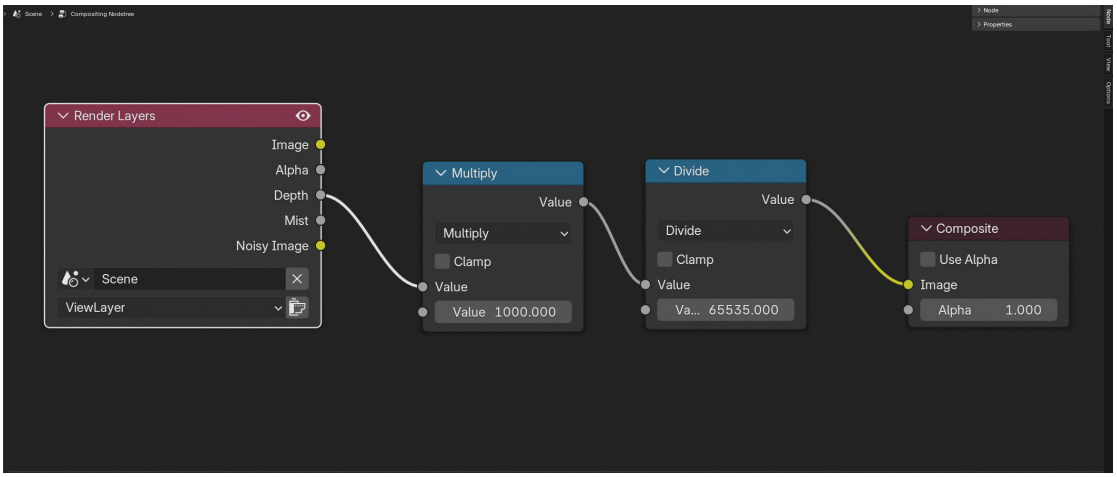


FIGURE 3.6: Blender's *Compositing* settings

Additionally, other settings were used as follows:

- in the *Render* settings, *Render Engine* was set to *Cycles*
- in the *View Layer* settings, Z-pass was added
- in the *Output* settings, *Color Depth* was set to 16-bit

The rest of the setting changes were included in a script available in the project repository [3].

3.1.2.1 *Perfect* camera model

The Blender camera was configured using RGB camera calibration data acquired from Orbbec Femto Mega camera. The parameters used were:

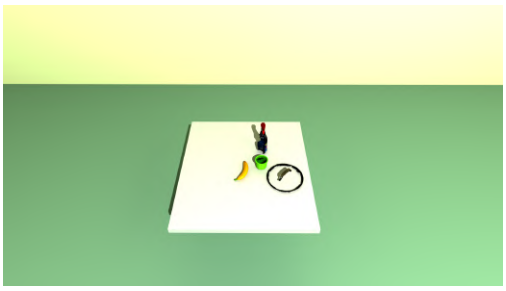
- focal length,
- resolution,
- camera principal point

Camera	**Resolution**	f_x**[px]**	f_y**[px]**	c_x**[px]**	c_y**[px]**
RGB	1920x1080	1121.560547	1121.922607	939.114746	534.70929

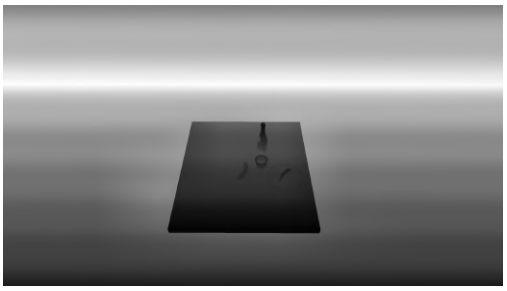
TABLE 3.1: RGB camera parameters

The FOV settings were obtained from Orbbec Femto Mega documentation [4] and set to 80° in horizontal axis and 51° in vertical axis. Real cameras feature *aligned* mode which allows to precisely overlay the depth image onto the RGB image, ensuring their spatial correspondence. In this mode, parameters of the depth camera are changed to match the RGB camera parameters. To recreate this phenomenon in an idealized way, the same calibration data was used for the depth camera. It allows for a perfect representation of the projection to RGB space used in the sensor's *aligned* mode. The representation is idealized, missing inaccuracies present in real world

sensors. Both camera objects were placed on the same coordinates in the simulated space. The parameters are shown in tab. 3.1.



(A) RGB image from the *perfect* camera model



(B) Depth image from the *perfect* camera model

FIGURE 3.7: Images from the *perfect* camera model

The depth image obtained from the *perfect* camera model is shown in fig 3.7b. It has RGB image (3.7a) properties, such as 1920x1080 resolution.

3.1.2.2 Proposed model

In Blender, the RGB camera settings were used as in the previous section. In this case, to accurately simulate the depth camera it is necessary to find its own parameters. The depth camera was configured using depth camera calibration data acquired from Orbbec Femto Mega camera. The parameters are shown in tab. 3.2.

Camera	**Resolution**	f_x**[px]**	f_y**[px]**	c_x**[px]**	c_y**[px]**
Depth	1024x1024	504.752167	504.695465	517.601746	508.529358

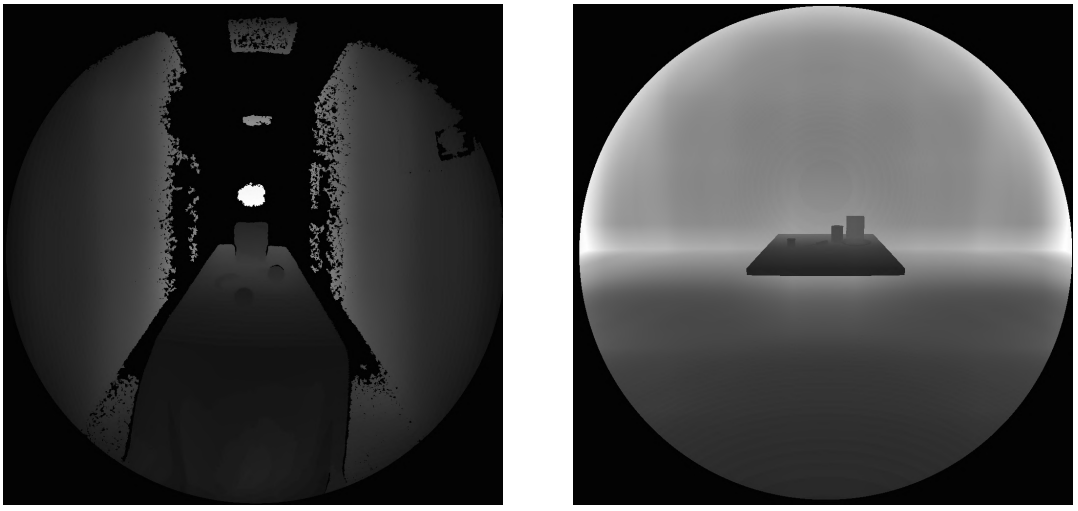
TABLE 3.2: Depth camera parameters

In addition, the depth camera type was set to *FISHEYE_EQUISOLID*, to simulate the fisheye effect observed in real world data acquired by the Femto Mega camera. This results in the depth camera having a wider, distorted field of view. The FOV settings were obtained from Orbbec Femto Mega documentation [4] and set to 120° in both vertical and horizontal axes. To accurately reproduce real world properties of the camera, it is essential to place depth and RGB cameras in different positions in 3D space, according to the transformation matrix acquired from the camera's calibration data. Transformation matrix parameters collected from Orbbec SDK are listed in tab. 3.3.

Parameter	Value
r_{11}	0.994052
r_{12}	0.002774
r_{13}	0.005608
r_{21}	-0.003367
r_{22}	0.994064
r_{23}	0.108743
r_{31}	-0.005723
r_{32}	-0.10876
r_{33}	0.994054
t_1	-0.032665543
t_2	-0.000986931
t_3	0.002863724

TABLE 3.3: Transformation parameters from RGB camera to depth camera

This way of placing the cameras allows for accurate recreation of the real world camera setup, including the inadequacies of the system. Fig. 3.8a presents depth image in a not-aligned mode obtained from Orbbec Femto Mega (3.8a). A circular shape and rounded edges can be seen on the real world image, implying the existence of the fisheye effect. Fig 3.8b shows the depth image with modeled physical properties, including lens distortions. It can be observed that the modeled image has a similar shape to the real-camera one (3.8).



(A) Real camera depth image in not-aligned mode

(B) Rendered model depth image

FIGURE 3.8: Comparison between the real image and the rendered model image

3.1.3 Simulating occlusions

Occlusions are areas of missing or undefined pixels on the depth image, where the sensor cannot capture information about surfaces. Occlusions are a common source of incomplete or noisy data in RGB-D images and present challenges for tasks such as 3D reconstruction and object recognition.

Simulating this effect is a difficult task, because of the multitude of different phenomena which may cause the pixels to be dropped from the image, including reflectivity, angle of reflection, difference in the field of view of the RGB and depth cameras, the color of the objects, the temperature of the sensor, and many more. Additionally, each of these has a degree of randomness, where some of the data may be lost, while the rest is retained. Therefore, focus was placed on the characteristics which can be derived from artificially generated images - particularly the angle of reflection, difference in FOVs and the color of the simulated objects. A probability function was found experimentally, based on observations of images taken with a real camera.

3.1.3.1 Difference in FOVs

Field of View (FOV) defines how much of a scene a camera can capture. Differences in FOV between RGB and depth sensors can lead to misalignment and affect data quality.

The difference between the fields of view of RGB and depth cameras leads to the appearance of shadow-like artifacts on one side of the objects within the captured data. Usually, those can be seen on the right side of objects because of the positioning of the sensors. These artifacts arise during the projection of a depth map into RGB image space, a process required to align the two images for further processing or visualization. Specifically, due to the difference in FOV and relative positioning of RGB and depth cameras within the simulation environment, certain regions captured by the depth sensor fall outside the viewing angle of the RGB sensor. As a result, when depth data is mapped onto an RGB frame, these unmatched regions manifest as shadows or missing data. The illustrative diagram is presented in fig. 3.9. The light-blue area represents the region of missing depth data.

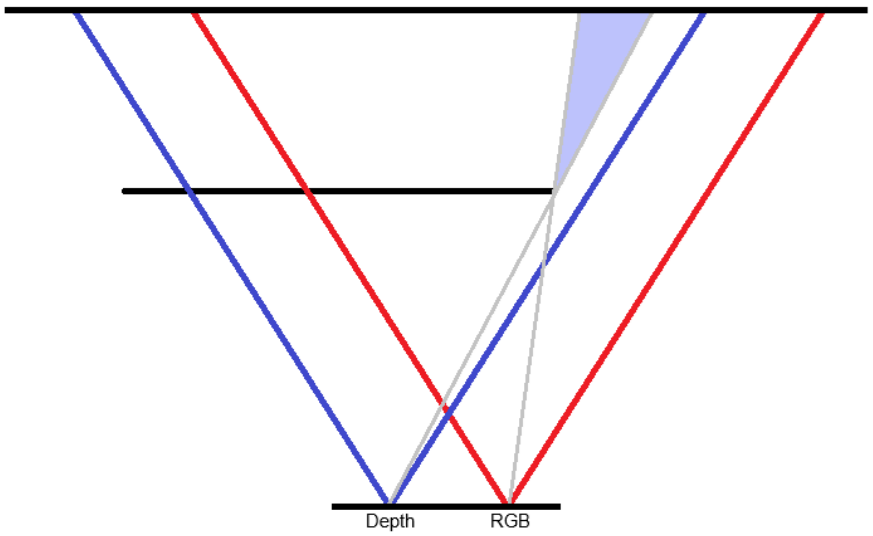
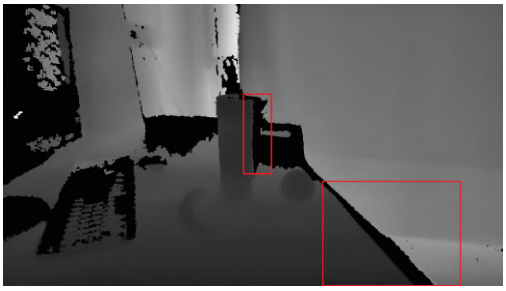


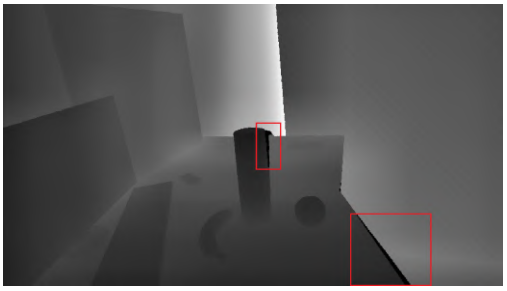
FIGURE 3.9: Diagram presenting occlusions present on aligned depth images, resulting from differences in FOVs

The projection process involves transforming a depth image using intrinsic and extrinsic parameters of both cameras. First, each pixel in the depth image is converted into a 3D point in the camera coordinate system using depth camera's intrinsic matrix. These 3D points are then transformed into the RGB camera coordinate system using known extrinsic calibration parameters. Finally, transformed points are projected onto the RGB image plane using the RGB camera's intrinsic matrix. Any areas where no corresponding depth information can be projected, due to the limited FOV, result in the previously mentioned artifacts, as seen in fig. 3.10.

These shadow regions are inherent to the geometric configuration and projection process and do not require additional post-processing. They reflect a realistic limitation of multi-modal sensor alignment.



(A) Occlusions caused by difference in FOVs in real-world data



(B) Occlusions caused by difference in FOVs in synthetic data

FIGURE 3.10: Occlusions caused by difference in FOVs

3.1.3.2 Angle of reflection

In depth cameras using AMCW, ToF or structured light, such as Orbbec Femto Mega (AMCW), depth is measured by projecting light beams onto the scene and analyzing their reflection. Using those measurement methods might cause occlusions when the projected beam does not return to the sensor, for example if incident angle is too high. The function that best represents the phenomenon that occurs in the actual camera was found experimentally and is presented by equation 3.1 and fig. 3.11. For angles lower than 60°, the drop probability is equal to 0. Between 60° and 75° probability grows quadratically, and for angles higher than 75° is equal to 1.

$$P_{drop} = (\frac{\theta - \theta_{min}}{\theta_0 - \theta_{min}})^2 for\ \theta \geq \theta_{min} \tag{3.1}$$

Where:

- θ - incident angle
- θ_{min} - minimum angle at which drops start
- θ_0 - angle at which all pixels are dropped

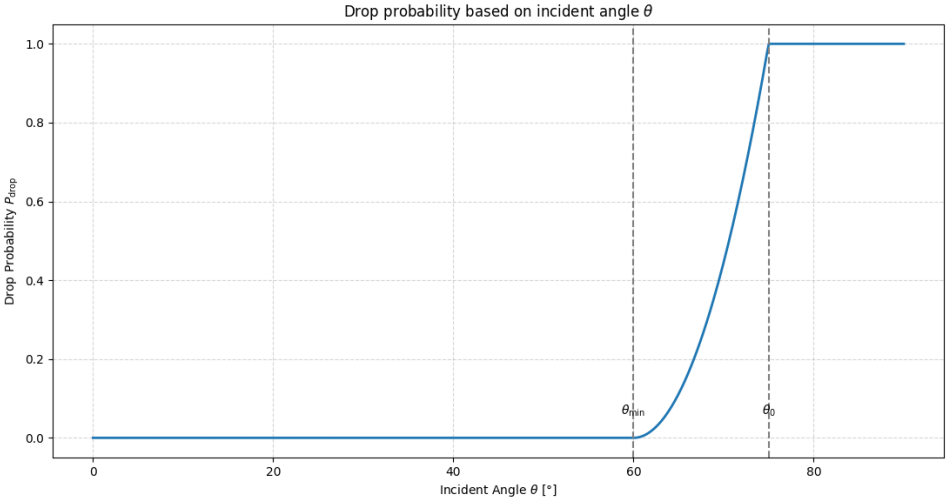


FIGURE 3.11: Drop probability based on incident angle θ

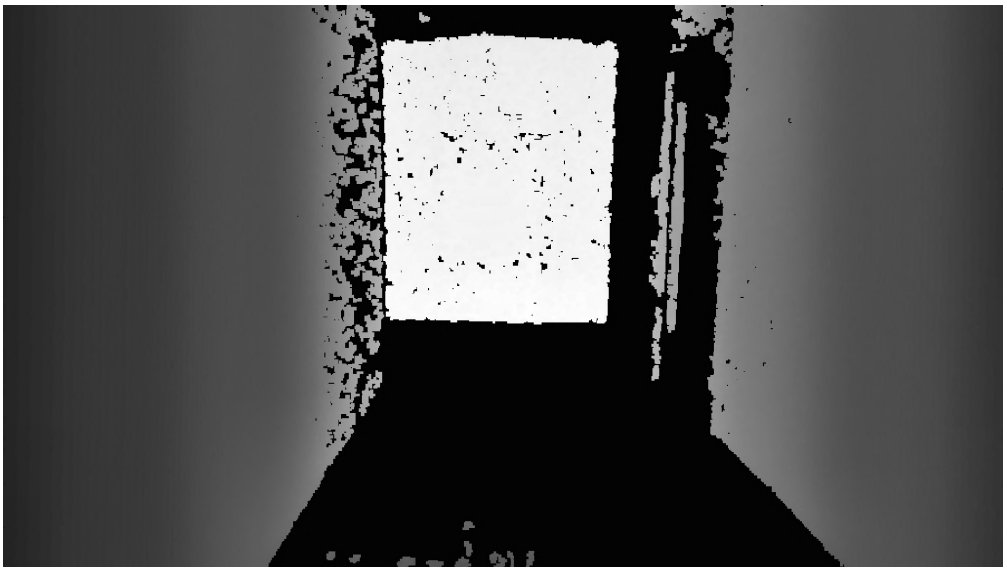


FIGURE 3.12: Pixel drops on the depth image from the Orbbec Femto Mega depending on the observation angle

Fig. 3.12 shows an image taken with real camera, in which pixel drops can be seen, particularly on the walls and the floor. The gaps on the wall are most likely a result of the high incident angle, while the ones on the floor are caused by a combination of the angle and the dark color of the flooring.

3.1.3.3 Color of the simulated objects

Another reason for light beams not returning to the sensor is the color of the objects. During the analysis of real world data, a correlation between low brightness of the objects on the image and higher amount of dropped pixels was observed. That is because darker objects absorb light more strongly, resulting in a weaker reflected signal. Additionally, the pixels seem to disappear

with rising incident angle at a much higher rate on dark objects. Fig. 3.13 shows a photo taken with the real camera, in which pixel drops can be observed due to:

- too high incident angle of the beam,
- low brightness of the objects.

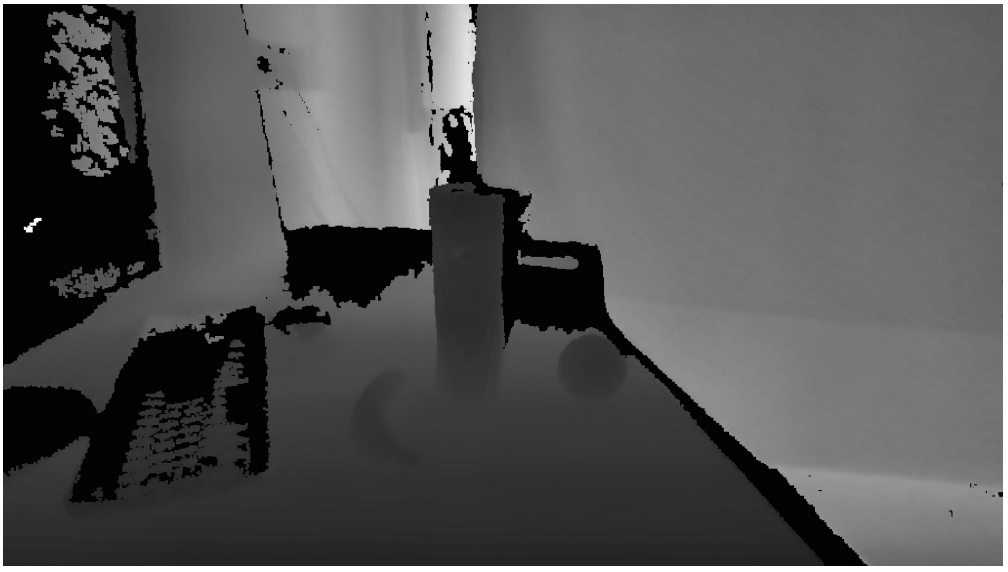


FIGURE 3.13: Real camera depth image

The influence of dark coloring is visible particularly on the monitor and keyboard on the left side of the image. The function describing this phenomenon was found experimentally and is presented in equation 3.2 and fig. 3.14.

$$P_{drop} = prob_{max} \cdot e^{-\frac{V}{V_{thresh}}} \cdot min(1, max((\frac{\theta}{\theta_{thresh}})^2, 0)) \tag{3.2}$$

Where:

- $prob_{max}$ - maximum probability of pixel drop
- V - brightness of pixel from HSV representation
- V_{thresh} - brightness threshold at which pixel drops stop
- θ - incident angle
- θ_{thresh} - incident angle threshold

Finally, probabilities were added together to calculate final pixel drop probability value for each pixel. Then, pixels were removed from the image based on that probability.

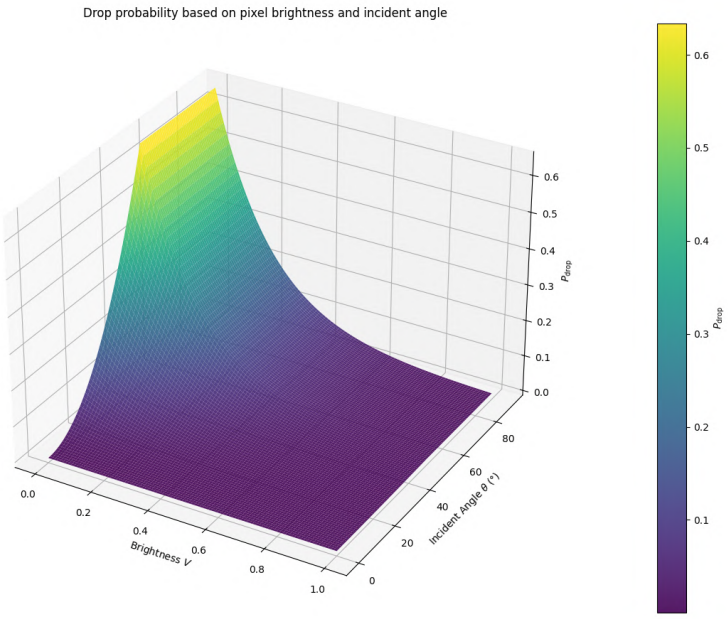
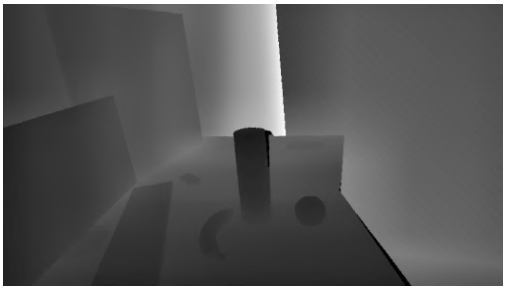
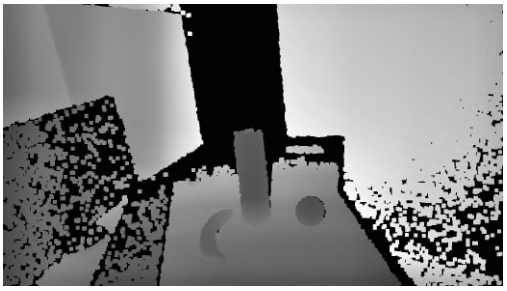


FIGURE 3.14: Drop probability based on brightness and incident angle θ for $prob_{max} = 0.65$



(A) Rendered image without pixel drop

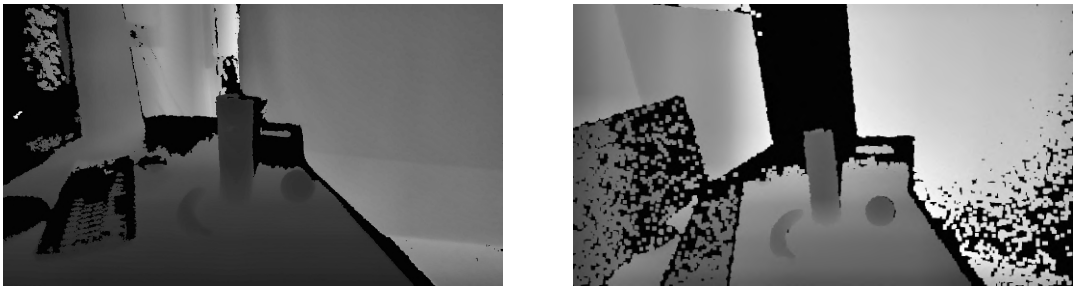


(B) Rendered image with pixel drop depending on the color and angle of observation

FIGURE 3.15: Rendered images with and without pixel drop depending on the color and angle of observation

Fig. 3.15a and fig. 3.15b show the differences between an image with and without pixel drop. It can be observed that image in fig. 3.15b is more similar to the photo taken with the real camera (3.16a). The differences between the images in fig. 3.16 are mainly due to inaccuracies in the scene modeling, especially in:

- inaccurate camera positioning,
- incorrect object placement,
- inaccurate color representation.



(A) Real camera photo

(B) Rendered image with pixel drop depending on the color and angle of observation

FIGURE 3.16: Comparison between the real photo and rendered image with pixel drop depending on the color and angle of observation

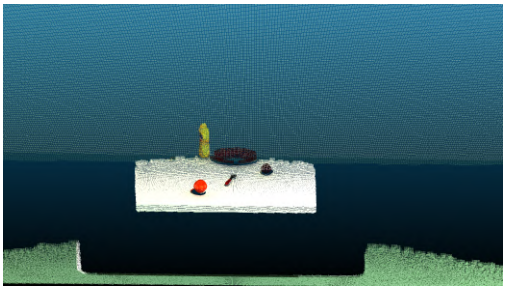
3.1.4 Simulating measurement noise

In the proposed model, two kinds of measurement noise were included:

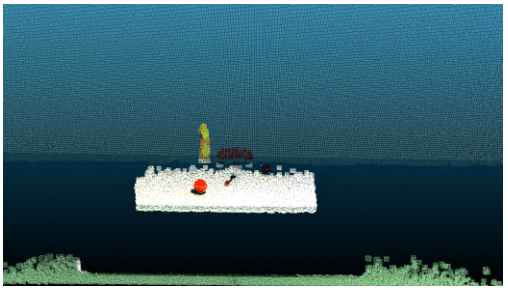
- axial noise - noise, which increases with the distance from the camera,
- lateral noise - noise which appears on the edges of surfaces and grows with the increase of the angle between the camera and the given plane.

Noise caused by factors such as the impact of sensor temperature or sunlight was not taken into consideration due to difficulty in modeling and overall low influence on the final image. Due to lack of information for Orbbec Femto Mega and newer Kinect sensors, such as Azure, formulas for axial 2.1 and lateral 2.3 noise for Kinect v2 were implemented as described in [24].

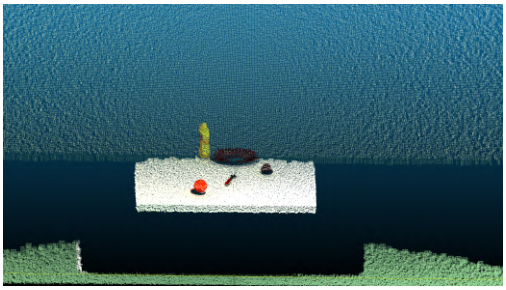
Fig. 3.17 shows the comparison between the point clouds before and after adding different types of noise. It can be observed how lateral 3.17b and axial 3.17c noise influence the patterns visible in the point cloud differently. Their combination results in a different pattern which can be seen in 3.17d.



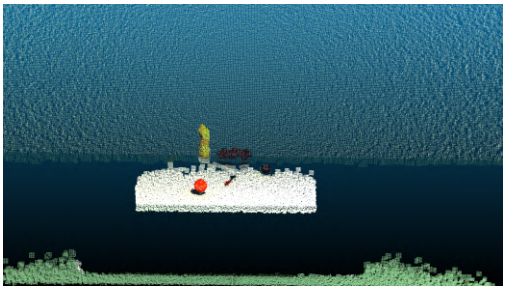
(A) Point cloud with pixel drops before adding noise



(B) Point cloud with added lateral noise



(C) Point cloud with added axial noise



(D) Point cloud with both lateral and axial noise added

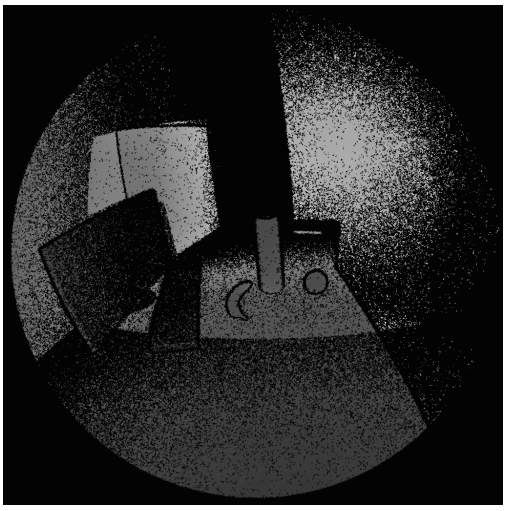
FIGURE 3.17: Comparison between the images of point cloud representations without and with measurement noise.

3.1.5 Projection of the depth image into RGB space

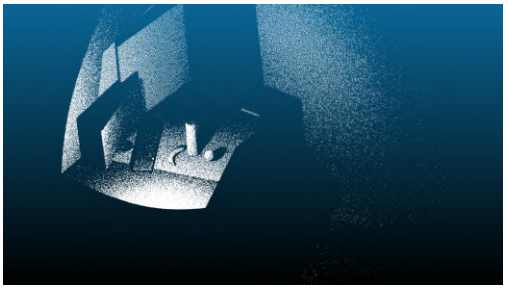
In many cases, the depth images used in real-world scenarios are not the raw depth images provided by the sensor, but instead *aligned* images, made by projecting the depth data into RGB camera space. This way of representing depth has many advantages, the most important being its easy, one-to-one relation with corresponding color image. To achieve that effect, following pipeline was prepared:

- converting raw depth image to 3D point cloud using depth camera intrinsic matrix K,
- transforming a point cloud from the depth camera coordinate frame to RGB camera coordinate frame,
- projecting 3D points onto image plane using RGB camera intrinsic matrix K.

If multiple points were projected onto the same pixel, only the point with the smallest depth value (the closest point) is retained.

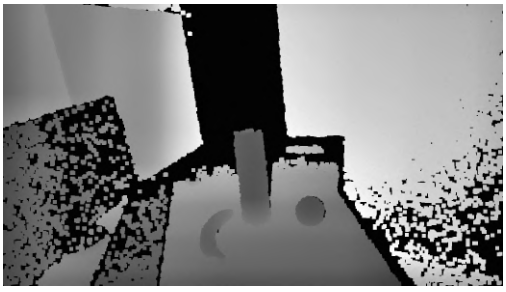


(A) Unaligned depth image with simulated inaccuracies

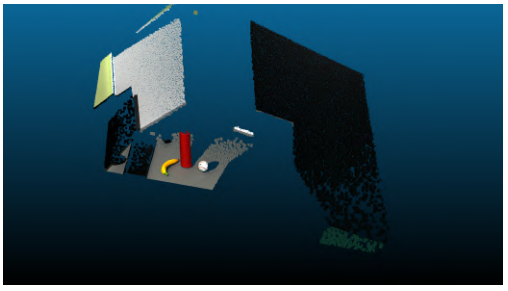


(B) Point cloud before projection

FIGURE 3.18: 2D and 3D representation of unaligned depth



(A) Aligned depth image with simulated inaccuracies



(B) Point cloud after projection

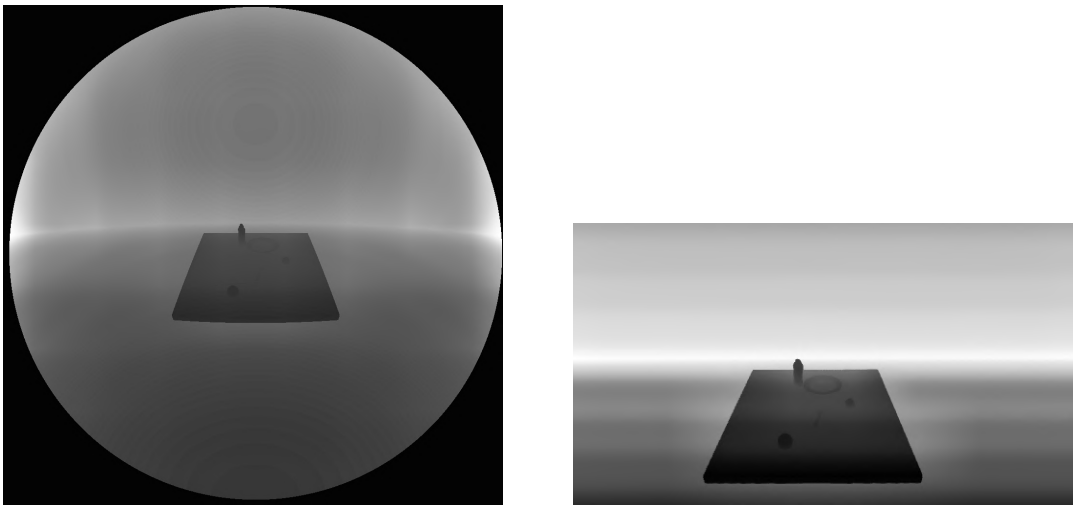
FIGURE 3.19: 2D and 3D representation of aligned depth

Fig. 3.18 shows the depth image and generated point cloud before the projection step. The rounded shape on most edges can be observed, as a result of the fisheye effect on the original depth image. The point cloud is not colored and the lack of interpolation can be seen, as pixels are much more sparse.

Fig. 3.19 presents the depth image and generated point cloud after the projection step. As a result of the interpolation, in 3.19b there are not as many gaps as in 3.18b. Colors of the points can be obtained from the RGB image due to the one-to-one relation between RGB and aligned depth images. The projection process visibly straightened the edges as well.

As a result of the projection process, a depth image in the same resolution as the RGB image is achieved. It was observed that pixels in the aligned image were much more sparse than on real aligned depth images. A nearest-neighbor interpolation was used to remedy that. The edges were retained through the interpolation process and data was then filtered to bias towards lower depth values.

Fig. 3.20 shows the differences between the unaligned depth image and the aligned depth image. It can be observed that the aligned image 3.20b has the shape of the RGB image.



(A) Unaligned depth (B) Aligned depth

FIGURE 3.20: Comparison between the unaligned depth image and the aligned depth image.

3.2 Novelty of the proposed system

The system proposed in this work introduces a new way of generating synthetic RGB-D data. It models many phenomena existing in real-world RGB-D imagery, which are often skipped in similar work, but are crucial to neural network training. For example, Zhang *et al.* [34] noted that their data lacked visible shadows. They also used a uniform Gaussian noise for their images, and the depth data was generated solely based on the RGB images. As such, a drop in average precision of their CNN from 98% to 68% was observed as they moved from evaluation on synthetic images to real-world inputs. Using 3D environments in Blender would allow for more accurate shadowing of RGB images. In addition, the explicit depth data available in the application combined with the model proposed in this work, which employs a more precise noise model and deliberate inaccuracies in data, could improve the network performance in real-world conditions.

Khan *et al.* [18] created a pipeline to generate accurate synthetic facial depth data. They created a scene using Blender, which consists of 3D human models, a light source and a camera in a fixed position. The main difference between their pipeline and the system proposed in this work is that it does not include transformations between RGB and depth camera and other inaccuracies that occur in the real camera. The system was tested on synthetic data. It can be observed that the proposed pipeline gave promising results. However, the performance of the networks on real-world inputs was not tested, and scores can be expected to decrease significantly.

The proposed system utilizes realistic discrepancy in sensor positioning between RGB and depth cameras, more precise noise model and generates artifacts such as missing pixels to more accurately simulate the data available from RGB-D cameras. This helps improve the generalization and robustness of neural networks trained on synthetic datasets, which is critical for their use in real-world scenarios.

Chapter 4

Experimental evaluation

This section presents the experimental evaluation conducted to validate the proposed approach. The goal of the experiments is to assess the performance and robustness of the developed system under various conditions. The test consists of two separate experiments. The first evaluation focuses on the comparison between quality of predictions of neural networks trained on synthetic data from the *perfect* and the proposed camera model. The second one focuses on measuring the influence of each modification step in our model on the final result. The test is performed on real world data, collected by Orbbec Femto Mega camera. An additional experiment was performed on synthetic testing data to validate both networks' performance.

4.1 Experimental setup

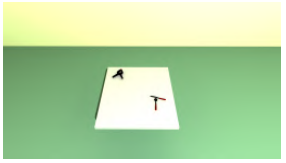
The experimental setup was designed to recreate high-resolution RGB image using U-Net architecture trained on low-resolution RGB images and high-resolution depth images. To perform the experiments, RGB images were converted to YCbCr color space. The method focuses on predicting high-resolution luminance channel. The chrominance components are obtained from the low-resolution YCbCr and upsampled. The channels are merged, in order to reconstruct the final high-resolution color image.

The experiments were conducted using a custom, synthetically generated datasets, with and without modifications of the proposed camera model. It consisted of:

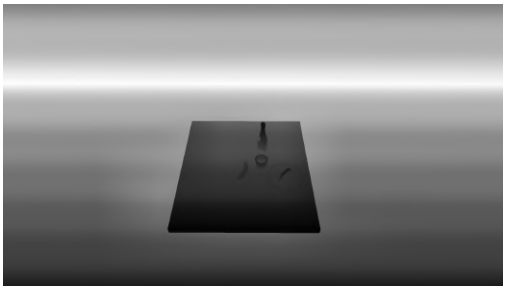
- high-resolution RGB images (1920x1080),
- corresponding depth images (1920x1080),
- downsampled low-resolution RGB images (256x144).

Training data examples are shown in fig. 4.1. In reality, low resolution RGB images are 7.5 times smaller than high resolution RGB images, but for the purpose of visualization they have been enlarged.

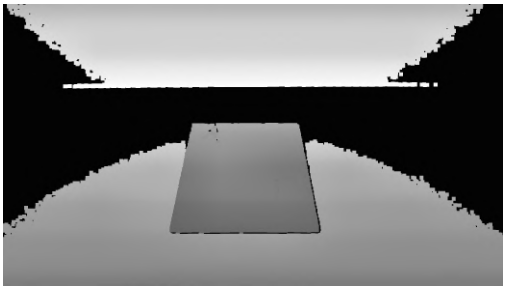
(A) Low resolution RGB image from the *perfect* camera model



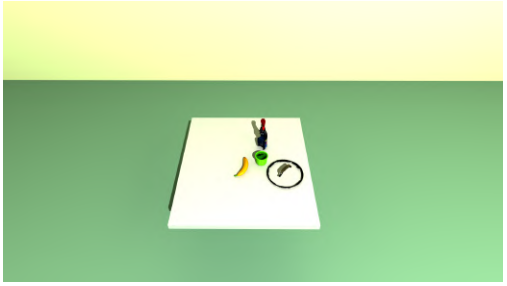
(B) Low resolution RGB image from the proposed camera model



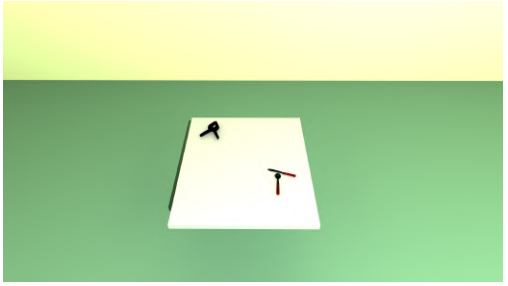
(C) Depth image from the *perfect* camera model



(D) Depth image from the proposed camera model



(E) High resolution RGB image from the *perfect* camera model



(F) High resolution RGB image from the proposed camera model

FIGURE 4.1: Training data examples

The dataset was augmented via:

- horizontal flipping,
- vertical flipping,
- 90° rotation,
- 180° rotation,
- 270° rotation.

It allowed for increasing the variability in the datasets. The metrics used to evaluate the performance of the tested networks were:

- PSNR - Peak Signal-to-Noise Ratio,
- SSIM (Y) - Structural Similarity Index measured on the luminance channel,
- SSIM (RGB) - Structural Similarity Index measured on the full RGB image channels,

- MAE - Mean Average Error,
- RMSE - Root Mean Square Error.

Two tests were performed to validate the trained networks, one on the real-world data obtained with Orbbec Femto Mega and one on synthetic data from the *perfect* camera model. Scripts used for these experiments are available at [2].

4.2 Experiments

4.2.1 Comparison between neural network trained on data from the proposed camera model and the *perfect* camera model

The main goal of the first experiment is to evaluate if modified synthetic depth images obtained from the proposed camera model are better for neural network training than synthetic data from the *perfect* camera model. To measure the impact of accurate camera modeling on the network performance, the following test on real-world data was conducted. Two U-Net network were trained.

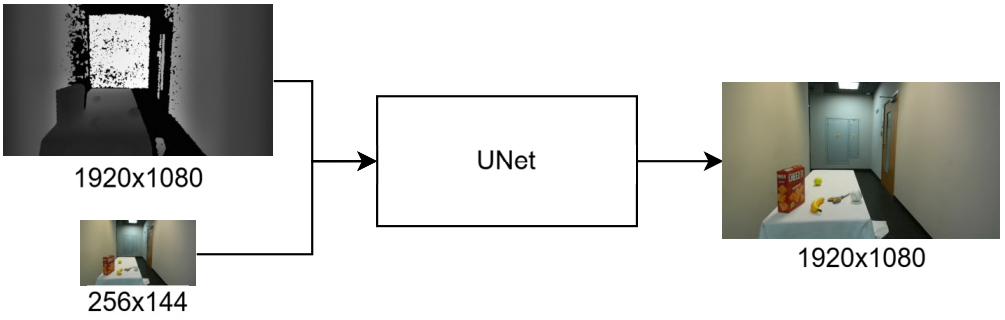


FIGURE 4.2: Overview of the neural network inputs and output during the testing process

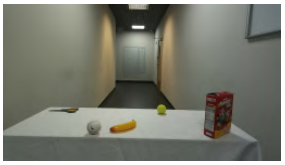
Fig. 4.2 represents neural network inputs and output during the testing process. The network takes low resolution RGB images and high resolution depth images to produce a high resolution RGB output.

The evaluation metrics are shown in tab. 4.1.

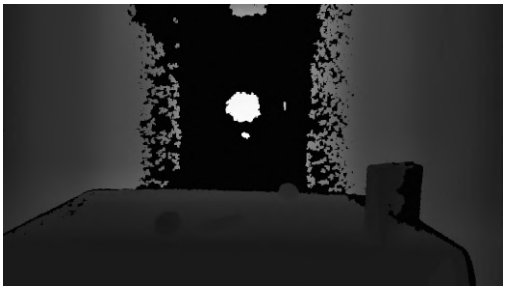
Input data	**PSNR[dB]** ↑	**SSIM(Y)** ↑	**SSIM** ↑	**MAE** ↓	**RMSE** ↓
Perfect model	30.627	0.9276	0.9246	0.0109	0.0306
Proposed model	**30.720**	**0.9285**	**0.9259**	**0.0105**	**0.0304**

TABLE 4.1: Results for different source of input data

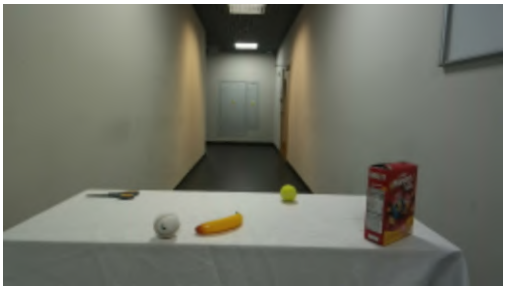
The results indicate that neural network trained on the data from the proposed camera model performs better on the images obtained from the real camera. All of examined metrics achieved better results, but the difference is not significant.



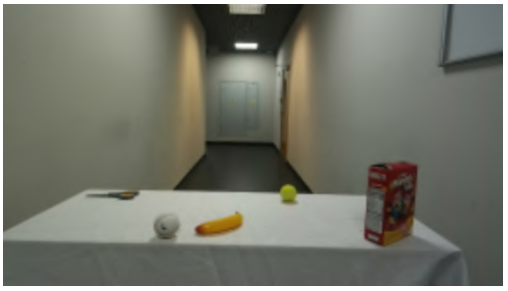
(A) Low resolution RGB image from Orbbec Femto Mega



(B) Depth image from Orbbec Femto Mega



(C) High resolution RGB prediction from neural network trained on data from the *perfect* camera model



(D) High resolution RGB prediction from neural network trained on data from the proposed camera model

FIGURE 4.3: High resolution RGB predictions

Fig. 4.3 presents predictions obtained from the neural networks trained on data from the *perfect* camera model 4.3c and the proposed camera model 4.3d. Due to minor differences between results, there is a possibility that the neural network learned to upscale the low resolution RGB image without using any depth information. In order to investigate that, an additional experiment was performed. A scaling factor λ was added to lower the importance of the down-scaled RGB data. To achieve that, the low-resolution RGB image was converted to YCbCr color space and its luminance channel was multiplied by different values (0.5, 0.75 and 1.0).

4.2.2 Comparison of the performance of the neural networks for different λ parameter values

This experiment is an extension of the previous one. Its goal is to validate the performance of the proposed model by manipulating the importance of depth data in the process of training a neural network. Multiple U-Net networks were trained for different λ parameter values. The λ parameter is a scaling factor used to modify the Y channel of the low-resolution YCbCr image before inputting it into the network. This allows for controlling the strength of the luminance channel in comparison to the depth signal, which in turn allows for controlling the influence that the depth data will have on the final prediction. The λ parameter enforces the network's reliance on the depth map, preventing it from focusing only on the low-resolution YCbCr data.

For each λ value two neural networks were trained - one on synthetic data from the *perfect* camera model (4.1a, 4.1c, 4.1e) and one on synthetic data from the proposed camera model (4.1b, 4.1d, 4.1f). The test was conducted on the images obtained from Orbbec Femto Mega (4.3a, 4.3b).

λ	**Input data**	**PSNR[dB]** $\uparrow$	**SSIM(Y)** $\uparrow$	**SSIM** $\uparrow$	**MAE** $\downarrow$	**RMSE** $\downarrow$
0.5	*Perfect* model	17.130	0.8793	0.8758	0.0556	0.1438
	Proposed model	**30.750**	**0.9289**	**0.9267**	**0.0099**	**0.0303**
0.75	*Perfect* model	25.313	0.8487	0.8507	0.0254	0.0552
	Proposed model	**30.717**	**0.9281**	**0.9255**	**0.0103**	**0.0304**
1	*Perfect* model	30.627	0.9276	0.9246	0.0109	0.0306
	Proposed model	**30.720**	**0.9285**	**0.9259**	**0.0105**	**0.0304**

TABLE 4.2: Results for different source of input data and λ parameter value

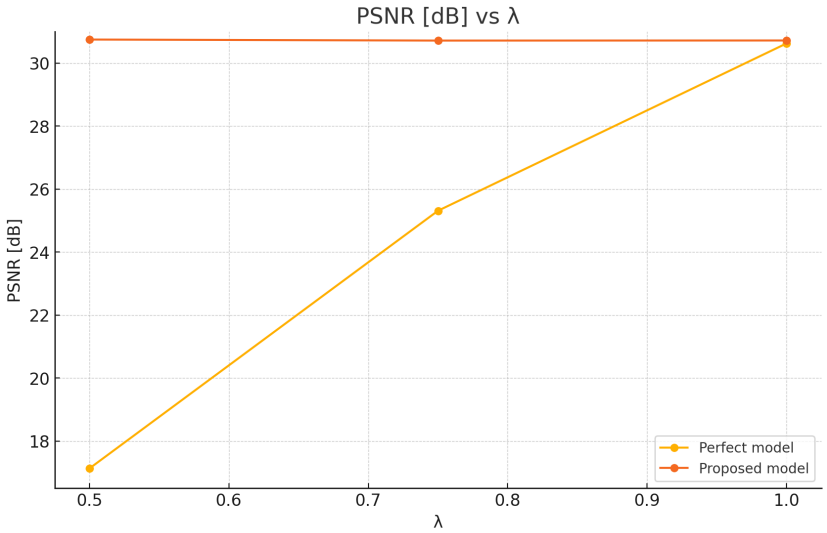
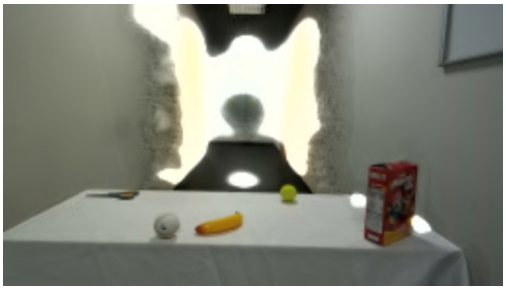


FIGURE 4.4: PSNR score for the neural network trained on data from the *perfect* camera model and for the neural network trained on data from the proposed camera model by λ value

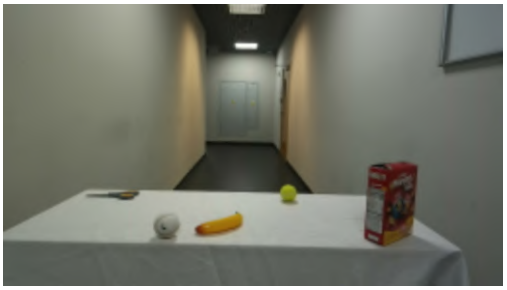
The results of conducted experiments are shown in tab. 4.2. Fig. 4.4 presents PSNR diagram for data from the *perfect* camera model and data from the proposed camera model by λ parameter. It can be observed that in most cases, neural network trained on data from the proposed model performed better. The greatest differences between metrics of networks trained on both datasets can be noted for lower λ values. Furthermore, for most λ values, the metrics for the proposed model outperform those for the *perfect* model. This shows that using properly modeled depth maps has a positive effect on the network's performance.

The neural network trained on data from the proposed model outperforms the network trained on the other dataset for λ values equal or higher than 0.5. The biggest difference between the results for the proposed model and the *perfect* model occurs for $\lambda = 0.5$. For this λ value, the proposed model also achieved the best metrics values. This means that, in tasks involving heavy use of the depth data, it is advantageous to use precise depth camera models, such as the model presented in this thesis.

Fig. 4.5 presents example predictions obtained from the neural networks. It can be observed that neural networks based on the *perfect* camera model have problems with the parts of the image

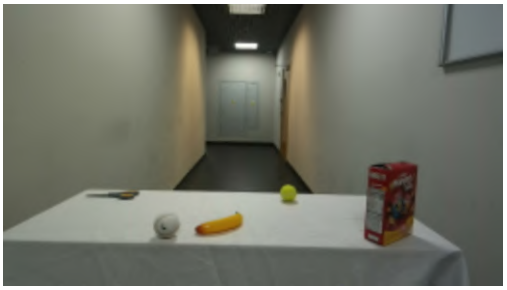


(A) *Perfect* camera model, $\lambda = 0.5$

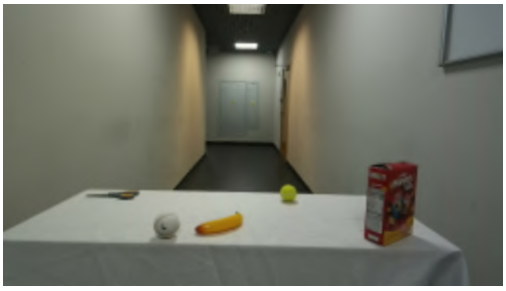


(B) Proposed camera model, $\lambda = 0.5$

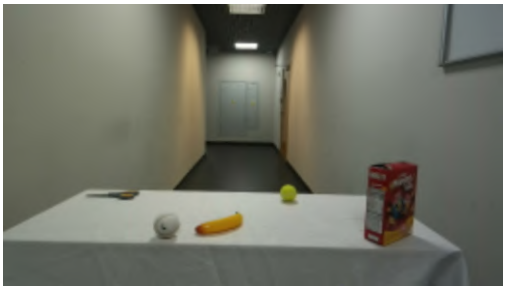
(C) *Perfect* camera model, $\lambda = 0.75$



(D) Proposed camera model, $\lambda = 0.75$



(E) *Perfect* camera model, $\lambda = 1.0$



(F) Proposed camera model, $\lambda = 1.0$

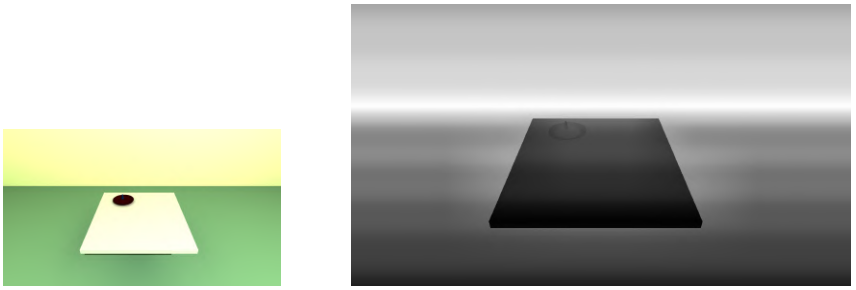
FIGURE 4.5: Example predictions based on data from the *perfect* and proposed camera models by λ value

where corresponding depth data is missing. The predictions based on data from the proposed model keep the resemblance to the original image for each tested λ value.

4.2.3 Comparison of the performance of the neural networks on data generated with the the *perfect* camera model

The goal of this experiment is to evaluate the performance of the trained neural networks on synthetic data obtained from the *perfect* camera model. The same networks used in the previous experiment were used and the conditions of the test remained identical. Real-world testing data was replaced with a set of generated image pairs which were not included in the networks' training. Examples of images from the test dataset are shown on fig. 4.6.

The main reason for this test was validating the correct performance of the *perfect* camera model on the same type of images. Additionally, it serves as a comparison between the networks in the context of synthetic imagery.



(A) Synthetic RGB image

(B) Synthetic depth image

FIGURE 4.6: Example synthetic RGB and depth images prepared as the test data

λ	**Input data** ↑	**PSNR[dB]** ↑	**SSIM(Y)** ↑	**SSIM** ↑	**MAE** ↓	**RMSE** ↓
0.5	*Perfect* model	34.013	**0.9833**	**0.9821**	0.0034	0.0200
	Proposed model	**34.040**	**0.9833**	**0.9821**	**0.0032**	**0.0199**
0.75	*Perfect* model	34.016	**0.9834**	0.9821	0.0038	0.0200
	Proposed model	**34.040**	0.9833	**0.9822**	**0.0032**	**0.0199**
1	*Perfect* model	**34.033**	**0.9833**	**0.9822**	**0.0032**	**0.0199**
	Proposed model	34.019	**0.9833**	**0.9822**	0.0040	0.0200

TABLE 4.3: Results for different input data and λ parameter value, tested on synthetic images

As seen on tab. 4.3, the performance of the network based on data from the *perfect* model is comparable to the one trained on images from the proposed model for most λ values. Only for $\lambda = 1.0$, the neural network based on the *perfect* model performs better. In comparison to the previous experiment, it can be observed that tests on the synthetic data gave better scores for both types of training data.

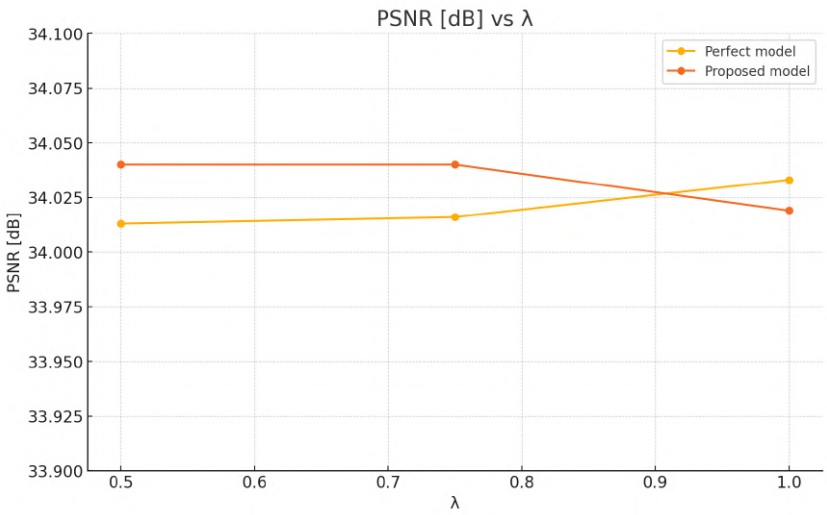
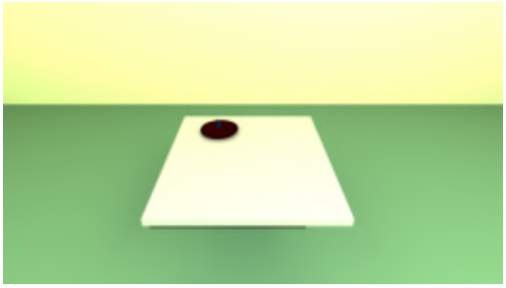
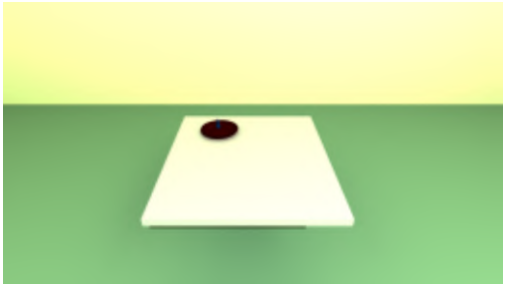


FIGURE 4.7: PSNR score for the neural network trained on data from the *perfect* camera model and for the neural network trained on data from the proposed camera model by λ, tested on synthetic images

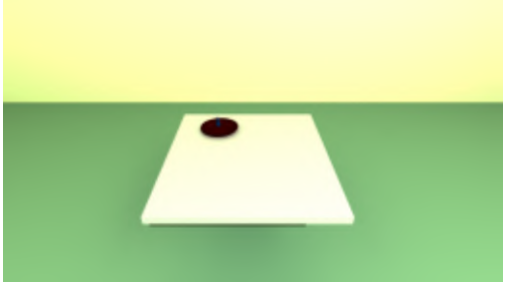
Fig. 4.7 presents PSNR diagram for the *perfect* and the proposed model data by λ parameter. Images shown in fig. 4.8 are the example predictions obtained from neural networks.



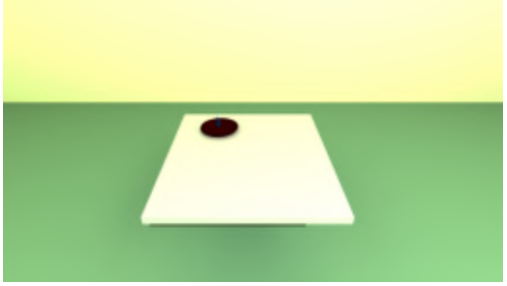
(A) *Perfect* camera model, $\lambda = 0.5$



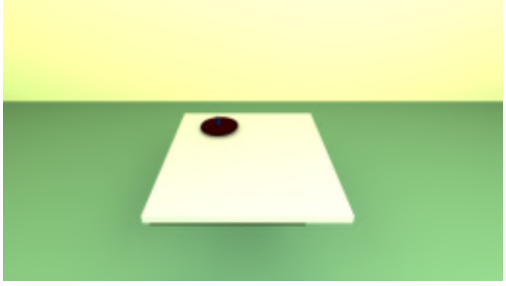
(B) Proposed camera model, $\lambda = 0.5$



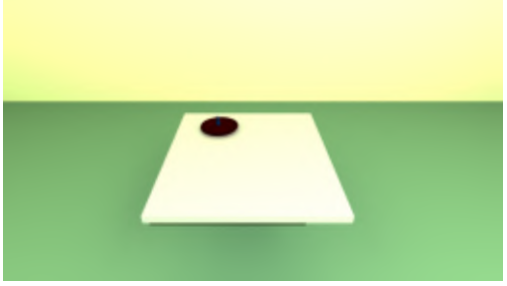
(C) *Perfect* camera model, $\lambda = 0.75$



(D) Proposed camera model, $\lambda = 0.75$



(E) *Perfect* camera model, $\lambda = 1.0$



(F) Proposed camera model, $\lambda = 1.0$

FIGURE 4.8: Example predictions based on data from the *perfect* and proposed camera models by λ value

4.2.4 Ablation study of the proposed depth camera model

The main goal of this experiment is to measure which steps of the proposed camera modeling pipeline have the biggest influence on the performance of the neural network. Multiple U-Net networks were trained on the datasets with different modifications applied. Five datasets were prepared, containing the depth images after each modification step:

- projection and interpolation,
- pixel drop,
- axial noise and lateral noise, separately
- all modifications.

The experiment was conducted for $\lambda = 0.5$. Testing data consisted of real world images from Orbbec Femto Mega (4.3a, 4.3b).

Input data	PSNR[dB] ↑	SSIM(Y) ↑	SSIM ↑	MAE ↓	RMSE ↓
Perfect camera model	17.130	0.8793	0.8758	0.0556	0.1438
Projection and interpolation	29.841	0.9206	0.9176	0.0139	0.0331
Pixel drop	30.738	0.9286	0.9262	0.0101	**0.0303**
Axial noise	30.743	0.9287	0.9263	0.0101	**0.0303**
Lateral noise	30.736	0.9285	0.9263	0.0102	**0.0303**
All modifications	**30.750**	**0.9289**	**0.9267**	**0.0099**	**0.0303**

TABLE 4.4: Results for each modification step

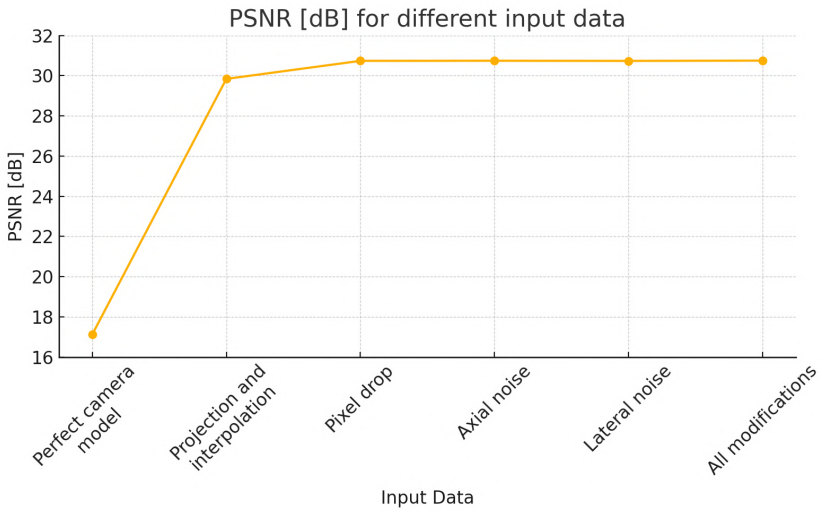
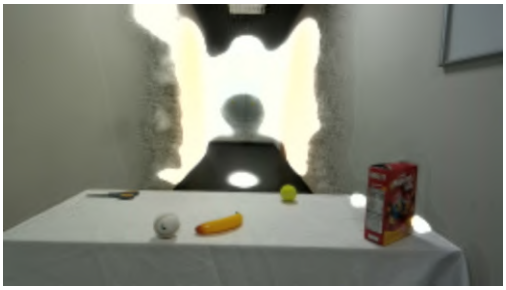


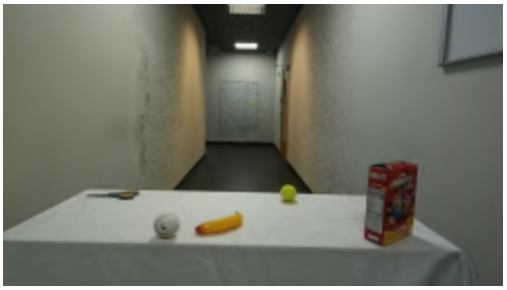
FIGURE 4.9: PSNR score for different modification steps

The results of the experiment are shown in tab. 4.4. Metrics for data from the the *perfect* camera model with $\lambda = 0.5$ were added for comparison. Fig. 4.9 presents PSNR values for different modification steps for $\lambda = 0.5$. As seen in the table, using camera parameters for projection of the depth data into RGB space, instead of depth map from the *perfect* camera model, increases the performance of the neural network significantly. Removing pixels improves the metrics, though each further modification has less effect on the overall prediction. Notably, using lateral noise separately from the axial noise has a negative effect on the outcome, though using both types of noise together has a bigger positive effect. It is worth adding that the network trained on the data with all modifications achieves the best scores for all metrics.

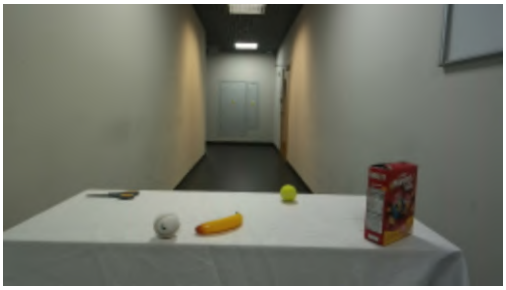
Fig. 4.10 present example predictions for each step of the modification process. The biggest difference can be noted between prediction from the neural network trained on the data from the *perfect* camera model (4.10a) and the neural network trained on the data with added projection and interpolation (4.10b). Another visible change occurs when pixel drop is added (4.10c). Further steps improve the output, but less significantly.



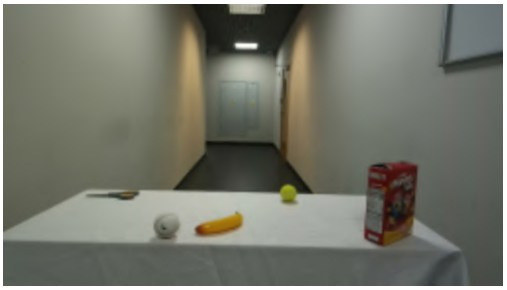
(A) Image from the *perfect* camera model



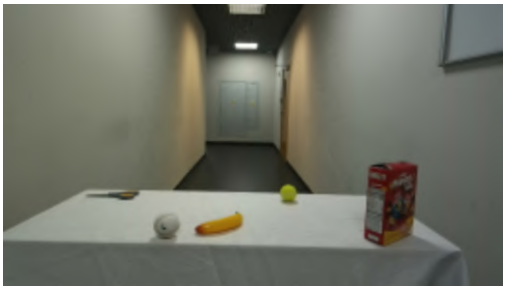
(B) Projection and interpolation



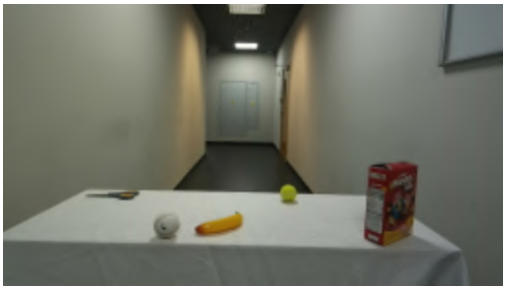
(C) Pixel drop



(D) Axial noise



(E) Lateral noise



(F) All modifications

FIGURE 4.10: Example predictions for each step of the modification process

4.3 Conclusions

The results of the experiments show that using depth images provided by the proposed model has a positive effect on the performance of neural networks trained using depth data. In most cases, the improvement is clearly visible, as can be seen on fig. 4.5. Evaluation shows that while a network trained on data from the *perfect* camera model performs just as well on synthetic testing data as the one trained on modified images, the resulting scores drop significantly when testing on real-world samples, as seen on tab. 4.2 and 4.3. Metrics also improve with decreasing λ values, which indicates that model would perform well in tasks more heavily relying on depth information. Fig. 4.10 and tab. 4.4 show that each step of the modification process is valuable and improves the final result.

Chapter 5

Conclusions

This chapter presents a summary of the entire work, including the proposed method and obtained results. It discusses the effectiveness of the developed approach and highlights the role of accurate synthetic data generation. The chapter also reflects on the limitations of the current solution and suggests directions for future research.

5.1 Summary

In this thesis, a model for accurate synthetic depth data generation was proposed. It differs from other works, because it more precisely recreates the artifacts and inaccuracies that occur in the real cameras.

A framework for generating RGB and depth images of 3D scenes modeled in Blender was presented. The camera model created in Blender included real device (Orbbec Femto Mega) parameters, lens structures, such as fisheye, and the geometric transformation between RGB and depth sensors. The images were rendered in pairs (RGB and depth image). Each pair was generated for a different, randomly created scene. YCB dataset was used to place objects on the scene. The images were subsequently subjected to postprocessing. This process included three steps: projection and interpolation, simulating occlusions and applying measurement noise. These transformations were based on the analysis of experimental data and information from literature. Projection and interpolation were necessary to overlay the depth information onto the RGB image. Occlusions were applied by randomly removing pixels from the depth image, based on the incident angle of the ray from the camera onto a surface and the color of objects on the image. The measurement (axial and lateral) noise was simulated using equations available in the literature.

To evaluate the proposed model, U-Net neural networks were prepared. Their purpose was to generate high-resolution RGB image, using low-resolution RGB and high-resolution depth inputs. They were trained on the data generated by the proposed and *perfect* RGB-D models. Two experiments were performed.

One experiment consisted of a comparison between predictions of neural networks trained on

data from the *perfect* camera model and data from the proposed camera model. A scaling factor λ was used to compare the performance of the networks based on the influence of the depth image. To evaluate the performance, PSNR, SSIM, RMSE and MAE metrics were used. The results showed that for most λ values, the proposed model performed better. Only for $\lambda = 0.25$, the model displayed worse scores. The experiment was then repeated for synthetic testing data from the *perfect* camera model, which showed the two networks achieving similar results.
The second experiment showed how each modification step changes the output image and metrics values. Networks were trained on data collected after each stage of the modifications present in the proposed model, and the λ was equal to 0.5. The results showed improvement after each step. The best scores were achieved with all modifications included.

Within this thesis, the following objectives were successfully achieved:

- a Blender scene, recreating an image from Orbbec Femto Mega camera, was prepared,
- a Blender script for rendering RGB and depth images from randomly generated scenes was prepared,
- lateral and axial noise models were implemented and added to the images,
- occlusions caused by high beam incident angle were modeled,
- occlusions caused by color of items within a scene were modeled,
- shadows caused by difference in RGB and depth camera FOVs were added,
- point clouds were generated from the preprocessed depth images,
- interpolation of point clouds was implemented,
- depth point clouds were projected into RGB image space,
- synthetic image datasets were generated, based on the proposed and *perfect* camera models,
- U-Net neural networks were trained on the prepared datasets for RGB image upscaling task,
- four experiments were prepared:
  - Comparison between neural network trained on data from the proposed camera model and from the *perfect* camera model,
  - Comparison of the performance of the neural networks for different λ parameter values,
  - Comparison of the performance of the neural networks on data generated with the *perfect* camera model,
  - Ablation study of the proposed depth camera model,
- data from the experiments was analyzed and described.

5.2 Conclusions

Based on the data collected during experiments, it was observed that networks trained on the data modified by the applied model performed better in tests using real-world images from Orbbec Femto Mega. This suggests that using more accurate depth image may be beneficial for tasks more reliant on depth information, such as detection task. It was proven that usage of the proposed camera model can boost a performance of neural networks in real-life scenarios.
Because upscaling RGB images is a task that can be achieved without using depth data, the performed experiment may not fully reflect the advantages of accurately modeling the depth image. However, the improvement in neural network performance after implementing a scaling factor λ indicates the benefits of the proposed camera model for tasks more heavily involving depth data.
While the networks trained on data from the *perfect* camera model performed just as well while tested on synthetic data, their performance decreased significantly when testing on real-world camera images. This shows the value of the proposed model for neural networks working in real-life conditions. It can be observed that even without applying the full model, simulating functions such as the projection of depth data into RGB space or occlusions could bring substantial improvement in the area of neural network training.

5.3 Future work

The proposed model was designed in a modular form, so it can be freely modified. It is possible to add further modifications or improve currently implemented ones. One possible future avenue of work is improving the proposed noise model. Since there is a lack of works analyzing the noise existent in Orbbec Femto Mega in recent literature, the model implemented one based on Kinect v2 research. A more accurate study of the Femto Mega noise could help improve the performance.
The implementation of the proposed model includes parameterization, so a user can freely change the camera parameters to simulate different device.
Based on experiments presented in this paper, the proposed model might be beneficial in use for different neural network tasks requiring depth data. Nevertheless, it requires more complex testing in a wider variety of use cases.

Bibliography

[1] https://www.blender.org/.

[2] . https://github.com/MaciejBr01/depth_guided_upscaling.

[3] . https://github.com/tomasz-naklicki/synthetic-rgbd-camera-model.

[4] https://www.orbbec.com/documentation-mega/femto-mega-hardware-specifications/.

[5] https://github.com/ankurhanda/simkinect.

[6] https://developer.nvidia.com/isaac/sim.

[7] https://www.ycbbenchmarks.com/object-models/.

[8] Jonathan T. Barron and Jitendra Malik. Intrinsic scene properties from a single rgb-d image. In *2013 IEEE Conference on Computer Vision and Pattern Recognition*, pages 17–24, 2013. doi: 10.1109/CVPR.2013.10.

[9] A. N. Belov, A. A. Zarubin, A. A. Savelieva, and A. V. Tarlykov. The use of synthetic data for training the neural network to classify the aircrafts. In *2019 Systems of Signals Generating and Processing in the Field of on Board Communications*, pages 1–4, 2019. doi: 10.1109/SOSG.2019.8706744.

[10] Y. Berdnikov and Dmitriy Vatolin. Real-time depth map occlusion filling and scene background restoration for projected-pattern-based depth camera. pages 1–4, 09 2011.

[11] Elia Bonetto, Chenghao Xu, and Aamir Ahmad. Grade: Generating realistic and dynamic environments for robotics research with isaac sim, 2024. URL https://arxiv.org/abs/2303.04466.

[12] Bernardus C. Bothma and Nicolaas Luwes. Exploring the use of synthetic training data for the classification of electronic components in artificial intelligence systems. In *2024 16th International Conference on Human System Interaction (HSI)*, pages 1–6, 2024. doi: 10.1109/HSI61632.2024.10613537.

[13] Daniel C, Juho Kannala, and Janne Heikkilä. Accurate and practical calibration of a depth and color camera pair. pages 437–445, 08 2011. ISBN 978-3-642-23677-8. doi: 10.1007/978-3-642-23678-5_52.

[14] Junhao Dong, Melvin Wong, Sihan Xia, and Joel Tay. Towards adversarially robust data-efficient learning with generated data. pages 1422–1424, 06 2024. doi: 10.1109/CAI59869.2024.00253.

[15] David Fiedler and Heinrich Müller. Impact of thermal and environmental conditions on the kinect sensor. In Xiaoyi Jiang, Olga Regina Pereira Bellon, Dmitry Goldgof, and Takeshi Oishi, editors, *Advances in Depth Image Analysis and Applications*, pages 21–31, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg. ISBN 978-3-642-40303-3.

[16] Georg Halmetschlager-Funek, Markus Suchi, Martin Kampel, and Markus Vincze. An empirical evaluation of ten depth cameras: Bias, precision, lateral noise, different lighting conditions and materials, and multiple sensor setups in indoor environments. *IEEE Robotics & Automation Magazine*, 26(1):67–77, 2019. doi: 10.1109/MRA.2018.2852795.

[17] Hyung-Min Jeon, Long Hoang Pham, Duong Nguyen-Ngoc Tran, Huy-Hung Nguyen, and Jae Wook Jeon. 3d synthetic image training and testing data for autonomous driving. In *2022 IEEE International Conference on Consumer Electronics-Asia (ICCE-Asia)*, pages 1–4, 2022. doi: 10.1109/ICCE-Asia57006.2022.9954858.

[18] Faisal Khan, Shubhajit Basak, Hossein Javidnia, Michael Schukat, and Peter Corcoran. High-accuracy facial depth models derived from 3d synthetic data. In *2020 31st Irish Signals and Systems Conference (ISSC)*, pages 1–5, 2020. doi: 10.1109/ISSC49989.2020.9180166.

[19] Kamil Lelowicz. Camera model for lens with strong distortion in automotive application. In *2019 24th International Conference on Methods and Models in Automation and Robotics (MMAR)*, pages 314–319, 2019. doi: 10.1109/MMAR.2019.8864659.

[20] Andreas Luber and Ralf Reulke. Validation of a polynomial camera model for generic cameras: Calibration of catadioptric, fish-eye, short- and long focal length cameras. In *2009 24th International Conference Image and Vision Computing New Zealand*, pages 179–184, 2009. doi: 10.1109/IVCNZ.2009.5378417.

[21] Tanwi Mallick, Partha Pratim Das, and Arun Kumar Majumdar. Characterizations of noise in kinect depth images: A review. *IEEE Sensors Journal*, 14(6):1731–1740, 2014. doi: 10.1109/JSEN.2014.2309987.

[22] David C. Marcu and Cristian Grava. The importance of data quality in training a deep convolutional neural network. In *2023 17th International Conference on Engineering of Modern Electric Systems (EMES)*, pages 1–4, 2023. doi: 10.1109/EMES58375.2023.10171785.

[23] Zhili Ng, Haozhe Wang, Zhengshen Zhang, Francis Tay Eng Hock, and Marcelo H. Ang Jr. Syntable: A synthetic data generation pipeline for unseen object amodal instance segmentation of cluttered tabletop scenes, 2025. URL https://arxiv.org/abs/2307.07333.

[24] Chuong V. Nguyen, Shahram Izadi, and David Lovell. Modeling kinect sensor noise for improved 3d reconstruction and tracking. In *2012 Second International Conference on 3D Imaging, Modeling, Processing, Visualization & Transmission*, pages 524–530, 2012. doi: 10.1109/3DIMPVT.2012.84.

[25] Harin Park, Inha Lee, Minje Kim, Hyungyu Park, and Kyungdon Joo. A benchmark dataset for collaborative slam in service environments. *IEEE Robotics and Automation Letters*, 9 (12):11337–11344, December 2024. ISSN 2377-3774. doi: 10.1109/lra.2024.3491415. URL http://dx.doi.org/10.1109/LRA.2024.3491415.

[26] Gang Peng, Xinhan Huang, Jian Gao, and Cheng Li. Camera modeling and distortion calibration for mobile robot vision. In *2008 7th World Congress on Intelligent Control and Automation*, pages 1657–1662, 2008. doi: 10.1109/WCICA.2008.4593167.

[27] Ritvik Singh, Jingzhou Liu, Karl Van Wyk, Yu-Wei Chao, Jean-Francois Lafleche, Florian Shkurti, Nathan Ratliff, and Ankur Handa. Synthetica: Large scale synthetic data for robot perception, 2024. URL https://arxiv.org/abs/2410.21153.

[28] T. Varga and H. Bunke. Comparing natural and synthetic training data for off-line cursive handwriting recognition. In *Ninth International Workshop on Frontiers in Handwriting Recognition*, pages 221–225, 2004. doi: 10.1109/IWFHR.2004.29.

[29] Luyao Wang and Guolin Liu. Three camera lens distortion correction models and its application. In *2022 3rd International Conference on Geology, Mapping and Remote Sensing (ICGMRS)*, pages 462–467, 2022. doi: 10.1109/ICGMRS55602.2022.9849271.

[30] Toshihiko Watanabe and Yuichi Saito. Practicable camera modeling technique applying fuzzy modeling for 3d sensing based on stereo vision. In *2013 Seventh International Conference on Sensing Technology (ICST)*, pages 434–439, 2013. doi: 10.1109/ICSensT.2013.6727690.

[31] Fan Wei, Guanghua Xu, Qingqiang Wu, Jiachen Kuang, Peiyuan Tian, Penglin Qin, and Zejiang Li. Azure kinect calibration and parameter recommendation in different scenarios. *IEEE Sensors Journal*, 22(10):9733–9742, 2022. doi: 10.1109/JSEN.2022.3166540.

[32] Jie Yu, Dirk Farin, Christof Krüger, and Bernt Schiele. Improving person detection using synthetic training data. In *2010 IEEE International Conference on Image Processing*, pages 3477–3480, 2010. doi: 10.1109/ICIP.2010.5650143.

[33] Cha Zhang and Zhengyou Zhang. Calibration between depth and color sensors for commodity depth cameras. pages 1–6, 07 2011. ISBN 978-3-319-08650-7. doi: 10.1109/ICME.2011.6012191.

[34] Shu Zhang, Cheng Song, and Rafael Radkowski. Setforge - synthetic rgb-d training data generation to support cnn-based pose estimation for augmented reality. In *2019 IEEE International Symposium on Mixed and Augmented Reality Adjunct (ISMAR-Adjunct)*, pages 237–242, 2019. doi: 10.1109/ISMAR-Adjunct.2019.00-39.